

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent	12413244
Kind Code	B2
Date of Patent	September 09, 2025
Inventor(s)	Haase; Paul et al.

Arithmetic encoder for arithmetically encoding and arithmetic decoder for arithmetically decoding a sequence of information values, methods for arithmetically encoding and decoding a sequence of information values and computer program for implementing these methods

Abstract

An encoding scheme is provided for arithmetically encoding a sequence of information values into an arithmetic coded bitstream by providing the bitstream with entry point information, allowing for resuming arithmetic decoding of the bitstream from a predetermined entry point onward. A respective decoding scheme is also provided. These encoding and decoding schemes provide more efficient encoding in view of the decoding speed.

Inventors: Haase; Paul (Berlin, DE), Kirchhoffer; Heiner (Berlin, DE), Mueller; Karsten (Berlin, DE), Schwarz; Heiko (Berlin, DE), Marpe; Detlev (Berlin, DE), Wiegand; Thomas (Berlin, DE)

Applicant: FRAUNHOFER-GESELLSCHAFT ZUR FOERDERUNG DER ANGEWANDTEN FORSCHUNG E.V. (Munich, DE)

Family ID: 72752840

Assignee: FRAUNHOFER-GESELLSCHAFT ZUR FOERDERUNG DER ANGEWANDTEN FORSCHUNG E.V. (Munich, DE)

Appl. No.: 18/296055

Filed: April 05, 2023

Prior Publication Data

Document Identifier	Publication Date
US 20230238982 A1	Jul. 27, 2023

Foreign Application Priority Data

EP

20200395

Oct. 06, 2020

Related U.S. Application Data

continuation parent-doc WO PCT/EP2021/077458 20211005 PENDING child-doc US 18296055

Publication Classification

Int. Cl.: H03M7/00 (20060101); H03M7/30 (20060101); H03M7/40 (20060101)

U.S. Cl.:

CPC H03M7/6023 (20130101); H03M7/4006 (20130101);

Field of Classification Search

CPC: H03M (7/6023); H03M (7/4006); H03M (7/70); H03M (7/6052)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
7592937	12/2008	Chang	341/107	H03M 7/4006
7630440	12/2008	Prakasam	382/247	H04N 19/91
9313514	12/2015	Misra	N/A	H04N 19/86
2012/0082218	12/2011	Misra	375/E7.243	H04N 19/46
2013/0223528	12/2012	Lim	375/240.12	H04N 19/436

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
2013543284	12/2012	JP	N/A
WO-2009047695	12/2008	WO	N/A

OTHER PUBLICATIONS

Anonymous, "Context-adaptive binary arithmetic coding", Wikipedia.org, Nov. 18, 2019, Available at URL: https://en.wikipedia.org/w/index.php?title=Context-adaptive_binary_arithmetic_coding&oldid=868799469. cited by applicant

Wiedemann, Simon, et al. "DeepCABAC: A universal compression algorithm for deep neural networks." IEEE Journal of Selected Topics in Signal Processing 14.4 (2020): 700-714. cited by applicant

ISO/IEC, "Information technology—Multimedia content description interface—Part 17: Compression of neural networks for multimedia content description and analysis", ISO/IEC 15938-17, Aug. 2022. cited by applicant

International Telecommunication Union, "Series H High efficiency video coding" ITU-T

Primary Examiner: Jeanglaude; Jean B

Attorney, Agent or Firm: PERRY + CURRIER INC.

Background/Summary

CROSS-REFERENCES TO RELATED APPLICATIONS (1) This application is a continuation of copending International Application No. PCT/EP2021/077458, filed Oct. 5, 2021, which is incorporated herein by reference in its entirety, and additionally claims priority from European Application No. EP 20 200 395.0, filed Oct. 6, 2020, which is incorporated herein by reference in its entirety. (2) The present application is concerned with arithmetically encoding a sequence of information values into an arithmetic coded bitstream, particularly by providing the bitstream with entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward. (3) Embodiments according to the invention are related to an arithmetic encoder for arithmetically encoding a sequence of information values. (4) Further embodiments according to the invention are related to an arithmetic decoder for arithmetically decoding a sequence of information values. (5) Further embodiments according to the invention are related to a method for arithmetically encoding a sequence of information values. (6) Further embodiments according to the invention are related to a method for arithmetically decoding a sequence of information values. (7) Further embodiments according to the invention are related to a computer program for performing methods for arithmetically encoding and decoding a sequence of information values. (8) Further embodiments according to the invention are related to a bitstream generated using an arithmetic encoder for arithmetically encoding a sequence of information values. (9) Further embodiments according to the invention are related to an arithmetic encoder for arithmetically encoding a neural network parameters. (10) Further embodiments according to the invention are related to an arithmetic decoder for arithmetically decoding a neural network parameters. (11) The invention can be applied to arbitrary data compression applications that involve the signaling of integer values like, for example, the compression of the parameters of neural networks.

BACKGROUND OF THE INVENTION

(1) A multitude of devices and methods for arithmetic encoding and decoding of values sequences are currently known. Particularly, a context-based adaptive binary arithmetic coding (CABAC) is widely used for encoding and decoding of a sequence of symbols. In a binarization stage of CABAC each symbol of such a sequence is converted into a sequence of one or more binary symbols (bins) and the concatenation of these bin sequences is arithmetically encoded into a bitstream. A context modeling stage then associates a probability estimate with each bin for arithmetic coding based on previously encoded bins and context information. The corresponding decoder has the same information available and can reproduce the same probability estimates in order to carry out arithmetic decoding.

(2) The known arithmetic decoding is, however, a highly sequential process that can hardly be parallelized. A decoding speed of CABAC decoder is thus limited by its serial coding procedure. (3) In view of the above, there is a desire to create a coding and decoding concept which enables several decoders to operate in parallel decoding different parts of a single bitstream, which leads to an improved decoding speed.

(4) Accordingly, it is the object of the present invention to provide an encoding concept which is more efficient in view of the decoding speed. This object is achieved by the subject matter of the pending independent claims.

(5) Further advantageous aspects are the subject of the dependent claims.

SUMMARY

(6) An embodiment may have an arithmetic encoder for arithmetically encoding a sequence of information values into an arithmetic coded bitstream, configured to: symbolize the information values into symbol strings so as to acquire a sequence of symbols; arithmetically encode the sequence of symbols by, for each symbol, selecting a subinterval out of a plurality of subintervals into which a current interval, which defines a current version of an coding state of the arithmetic encoder, is subdivided according to a probability estimate for the respective symbol, according to a symbol value of the respective symbol so as to acquire an updated version of the coding state of the arithmetic encoder, defined by the selected subinterval, for encoding a next symbol of the sequence of symbols, and renormalizing encoder-internal parameters which define the coding state under continuing the bitstream, provide the bitstream with entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward.

(7) Another embodiment may have an arithmetic decoder for arithmetically decoding a sequence of information values from a bitstream, configured to: derive from the bitstream entry point information; use the entry point information so as to resume arithmetically decoding the bitstream from a predetermined entry point onward by arithmetically decoding a sequence of symbols from the bitstream by, for each symbol of the bitstream: determining, based on a current version of a coding state of the arithmetic decoder, a subinterval out of a plurality of subintervals into which a current interval is subdivided according to a probability estimate for the respective symbol, and deduce, based on the selected subinterval, a symbol value of the respective symbol, and renormalizing and updating decoder-internal parameters which define the coding state by use of the bitstream and the selected subinterval so as to acquire an updated version of the coding state of the arithmetic decoder for decoding a next symbol of the sequence of symbols, and derive the information values from the sequence of symbols by desymbolization.

(8) Another embodiment may have a method of arithmetically encoding a sequence of information values into an arithmetic coded bitstream, comprising: symbolizing the information values into symbol strings so as to acquire a sequence of symbols; arithmetically encoding the sequence of symbols by subdividing a current interval, which defines a current version of an coding state of an arithmetic encoder, for each symbol, according to a probability estimate for the respective symbol, selecting a subinterval out of a plurality of the subintervals according to a symbol value of the respective symbol so as to acquire an updated version of the coding state of the arithmetic encoder, defined by the selected subinterval, for encoding a next symbol of the sequence of symbols, and renormalizing encoder-internal parameters which define the coding state under continuing the bitstream, providing the bitstream with entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward.

(9) Another embodiment may have a method of arithmetically decoding a sequence of information values from a bitstream, comprising: deriving from the bitstream entry point information; using the entry point information so as to resume arithmetically decoding the bitstream from a predetermined entry point onward by arithmetically decoding a sequence of symbols from the bitstream by, for each symbol of the bitstream: determining, a current version of an coding state of an arithmetic decoder, a subinterval out of a plurality of subintervals into which a current interval is subdivided according to a probability estimate for the respective symbol, and deducing, based on the selected subinterval, a symbol value of the respective symbol, and renormalizing and updating decoder-internal parameters which define the coding state by use of the bitstream and the selected subinterval so as to acquire an updated version of the coding state of the arithmetic decoder for decoding a next symbol of the sequence of symbols, and deriving the information values from the

sequence of symbols by desymbolization.

(10) Another embodiment may have non-transitory digital storage medium having a computer program stored thereon to perform the inventive methods when said computer program is run by a computer.

(11) Another embodiment may have bitstream generated using an arithmetic encoder according to the invention.

(12) An embodiment according to the invention creates an arithmetic encoder for arithmetically encoding a sequence of information values into an arithmetic coded bitstream. The arithmetic encoder is configured to symbolize the information values into symbol strings so as to obtain a sequence of symbols; arithmetically encode the sequence of symbols by, for each symbol, selecting a subinterval out of a plurality of subintervals into which a current interval, which defines a current version of an coding state of the arithmetic encoder, is subdivided according to a probability estimate for the respective symbol, according to a symbol value of the respective symbol so as to obtain an updated version of the coding state of the arithmetic encoder, defined by the selected subinterval, for encoding a next symbol of the sequence of symbols, and renormalizing encoder-internal parameters, e.g. parameters R and L, which define the coding state under continuing the bitstream, and provide the bitstream with entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward.

(13) This embodiment is based on the finding that providing the bitstream with the entry point information enables parallelizing of the arithmetic decoding, such as CABAC decoding, for example when several decoders operate in parallel decoding different parts of a single bitstream. This leads to a more efficient encoding and decoding concept, particularly to an improved decoding speed.

(14) According to an embodiment, the entry point information comprises information on an coding state of an arithmetic decoder, which occurs in the arithmetic decoder when the arithmetic decoder decodes the bitstream until the predetermined entry point. This allows using several decoders to parallelize decoding starting from several predetermined entry points.

(15) According to an embodiment, the arithmetic encoder is configured to perform the arithmetic decoding the bitstream so as to determine the information on the coding state of the arithmetic decoder. As an example, the arithmetic decoding may be executed after having finalized the arithmetic encoding. Alternatively, arithmetic decoding may be commenced in parallel with the arithmetic encoding, at a slight inter-bitstream-bit-distance between the bit position at which the arithmetic encoding continues to append bits and the bitstream begin at which the arithmetic decoding starts.

(16) According to an embodiment, the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the information on the coding state of the arithmetic decoder comprises a value of the pointer, e.g. which value is assumed by the pointer at the predetermined entry point.

(17) According to an embodiment, the arithmetic encoder is configured to perform the arithmetic decoding the bitstream and set the value of the pointer comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the pointer as manifesting itself in the arithmetic decoding the bitstream up to the predetermined entry point.

(18) According to an embodiment, the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the information on the coding state of the arithmetic decoder comprises a value of the interval width parameter, e.g. which value is assumed by the interval width parameter at the predetermined entry point.

(19) According to an embodiment, the arithmetic encoder is configured to perform the arithmetic decoding the bitstream and set the value of the interval width parameter comprised by the

information on the coding state of the arithmetic decoder to be equal to a current value of the interval width parameter as manifesting itself in the arithmetic decoding the bitstream up to the predetermined entry point. Alternatively the arithmetic encoder is configured to set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the interval width parameter as manifesting itself in the arithmetic encoding the sequence of symbols up to the predetermined entry point.

(20) According to an embodiment, the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the information on the coding state of the arithmetic decoder comprises a value of the pointer, e.g. which value is assumed by the pointer at the predetermined entry point, while being free of a value of the interval width parameter. In this embodiment the arithmetic encoder is configured to set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a predetermined value and use the predetermined value for the interval width parameter when resuming the arithmetic encoding the sequence of symbols from the predetermined entry point onwards. Alternatively, in this embodiment the arithmetic encoder is configured to preliminarily interrupt the arithmetic encoding the sequence of symbols by arithmetically encoding a symbol of a predetermined symbol value immediately ahead the predetermined entry point before resuming the arithmetic encoding the sequence of symbols from the predetermined entry point onwards, and set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the interval width parameter as manifesting itself in the arithmetic encoding the sequence of symbols up to the predetermined entry point, including the symbol of the predetermined symbol value.

(21) According to an embodiment, the entry point information comprises a bitstream pointer to a predetermined bit in the bitstream, which is up to be read next after the resuming the arithmetic decoding the bitstream from the predetermined entry point onwards.

(22) According to an embodiment, the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a begin, e.g. a leading end from which onwards the bitstream is to be decoded, of the bitstream.

(23) According to an embodiment, the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a trailing end of a run of leading bits of the bitstream based on which an coding state, e.g. the pointer into the interval, of the arithmetic decoder for performing the arithmetic decoding of the bitstream is to be initialized.

(24) According to an embodiment, the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward, and the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a previous entry point. The entry point information, for example, comprises more than one information instantiations, namely one per entry point. Alternatively, the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a predefined bitstream position which is associated with the predetermined entry point. For instance, n .sup.th predefined bitstream position is associated with n .sup.th entropy point; the predefined bitstream positions might be located mutually equi-distanced.

(25) According to an embodiment, the arithmetic encoder is configured to locate the predefined bitstream position as a point between consecutive ones of the sequence of values, in other words: syntactically, or by counting bits of the bitstream, or in other words: on bit level.

(26) According to an embodiment, the bitstream pointer to the predetermined bit in the bitstream is stored in units of bits, and/or an integer multiple n of bits, with $n > 1$, and $n = 8$, for example.

(27) According to an embodiment, a bit position in the bitstream of the previous entry point is signaled in the bitstream.

(28) According to an embodiment, the bitstream pointer is signaled in the bitstream differentially

with respect to a further bit stream pointer which is comprised by the entry point information for pointing to a further predetermined bit in the bitstream, which is up to be read next after a resuming the arithmetic decoding the bitstream from a preceding entry point onwards.

(29) According to an embodiment, the bitstream pointer is signaled in the bitstream using a variable length code.

(30) According to an embodiment, the bitstream pointer is signaled in the bitstream using an Exponential-Golomb code, preferably an unsigned Exponential-Golomb code.

(31) According to an embodiment, the predetermined entry point is a third or any of entry points following after the third entry point relative to the begin of the bitstream, and the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of a difference between the offset relative to the previous entry point and an offset of the previous entry point relative to a further entry point preceding the previous entry point.

(32) According to an embodiment, the bitstream pointer is signaled in the bitstream using a signed Exponential-Golomb code.

(33) According to an embodiment, the bit position in the bitstream of the previous entry point is signaled in the bitstream using an unsigned Exponential-Golomb code.

(34) According to an embodiment, an exponential Golomb code parameter for the Exponential-Golomb code is a value of 11.

(35) According to an embodiment, the bitstream pointer is signaled in the bitstream using a signed Exponential-Golomb code.

(36) According to an embodiment, an exponential Golomb code parameter for the Exponential-Golomb code is a value of 7.

(37) According to an embodiment, the arithmetic encoder is configured to use context-adaptive arithmetic coding for arithmetically encoding the sequence of symbols, including selecting, for a context-adaptively encoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols. The entry point information is indicative of, for each of a set of one or more predetermined context models, a predetermined probability estimate for the respective predetermined context model. The arithmetic encoder is configured to use the predetermined probability estimate in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model.

(38) According to an embodiment, the arithmetic encoder is configured to use context-adaptive arithmetic coding for arithmetically encoding the sequence of symbols, including selecting, for a context-adaptively encoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols. The arithmetic encoder is configured to, for each of a set of one or more predetermined context models, set the probability estimate for the respective predetermined context model at the predetermined entry point to a default state and the arithmetic encoder is configured to use the default state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model.

Alternatively in this embodiment the arithmetic encoder is configured to, for each of a set of one or more predetermined context models, set the probability estimate for the respective predetermined context model at the predetermined entry point to a saved state manifesting itself at a predetermined condition during arithmetically encoding the sequence of symbols ahead the predetermined entry point and the arithmetic encoder is configured to use the saved state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model.

(39) According to an embodiment, the arithmetic encoder is configured to use dependent

quantization for deriving the sequence of information values from a sequence of unquantized values using state machine, wherein the entry point information comprises a quantization state manifesting itself in the state machine up to the predetermined entry point.

(40) According to an embodiment, the symbols are bins and the symbolization is a binarization.

(41) According to an embodiment, the information values is a sequence of syntax elements which represent a video.

(42) According to an embodiment, the information values are neural network parameters.

(43) An embodiment according to the invention creates an arithmetic decoder for arithmetically decoding a sequence of information values from a bitstream. The arithmetic decoder is configured to: derive from the bitstream entry point information; use the entry point information so as to resume arithmetically decoding the bitstream from a predetermined entry point onward by arithmetically decoding a sequence of symbols from the bitstream by, for each symbol of the bitstream: determining, based on a current version of a coding state of the arithmetic decoder, a subinterval out of a plurality of subintervals into which a current interval is subdivided according to a probability estimate for the respective symbol, and deduce, based on the selected subinterval, a symbol value of the respective symbol, and renormalizing and updating decoder-internal parameters, e.g. R and V, which define the coding state by use of the bitstream and the selected subinterval so as to obtain an updated version of the coding state of the arithmetic decoder for decoding a next symbol of the sequence of symbols, and derive the information values from the sequence of symbols by desymbolization.

(44) The arithmetic decoder according to this embodiment is based on the same considerations as the arithmetic encoder described above.

(45) According to an embodiment, the arithmetic decoder is configured to use the entry point information so as to determine a starting version of the coding state of the arithmetic decoder and use the starting state for commencing arithmetically decoding the bitstream from the predetermined entry point onwards.

(46) According to an embodiment, the coding state of the arithmetic decoder is defined by the decoder-internal parameters which include an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the arithmetic decoder is configured to derive from the entry point information a starting value of the pointer, e.g. which value is assumed by the pointer at the predetermined entry point, using which the arithmetically decoding the bitstream from the predetermined entry point onwards is commenced.

(47) According to an embodiment, the coding state of the arithmetic decoder is defined by the decoder-internal parameters which include an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the arithmetic decoder is configured to derive from the entry point information a starting value of the interval width parameter, e.g. which value is assumed by the interval width parameter at the predetermined entry point, using which the arithmetically decoding the bitstream from the predetermined entry point onwards is commenced.

(48) According to an embodiment, the coding state of the arithmetic decoder is defined by the decoder-internal parameters which include an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the arithmetic decoder is configured to derive from the entry point information a starting value of the pointer, e.g. which value is assumed by the pointer at the predetermined entry point, using which the arithmetically decoding the bitstream from the predetermined entry point onwards is commenced and to set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a predetermined value and use the predetermined value for the interval width parameter for resuming the arithmetic decoding the sequence of symbols from the predetermined entry point onwards.

(49) According to an embodiment, the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward and the arithmetic decoder is

configured to preliminarily interrupting the arithmetic decoding the sequence of symbols at a subsequent predetermined entry point by arithmetically decoding a symbol of a predetermined symbol value immediately ahead the subsequent predetermined entry point before resuming the arithmetic decoding the sequence of symbols from the subsequent predetermined entry point onwards.

(50) According to an embodiment, the arithmetic decoder is configured to derive from the entry point information a bitstream pointer to a predetermined bit in the bitstream and use the predetermined bit as a bit to be read next after resuming the arithmetic decoding the bitstream from the predetermined entry point onwards.

(51) According to an embodiment, the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a begin, e.g. a leading end from which onwards the bitstream is to be decoded, of the bitstream.

(52) According to an embodiment, the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a trailing end of a run of leading bits of the bitstream based on which the arithmetic decoder is configured to initialize the coding state, e.g. the pointer into the interval, of the arithmetic decoder in case of performing the arithmetic decoding of the bitstream from the begin of the bitstream onwards.

(53) According to an embodiment, the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward, and the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a previous entry point or relative to a predefined bitstream position which is associated with the predetermined entry point. For instance, $n \cdot \text{sup.th}$ predefined bitstream position is associated with n th entropy point; the predefined bitstream positions might be located mutually equi-distanced.

(54) According to an embodiment, the arithmetic decoder is configured to locate the predefined bitstream position as a point between consecutive ones of the sequence of values, in other words: syntactically, or by counting bits of the bitstream, or in other words: on bit level.

(55) According to an embodiment, the bitstream pointer to the predetermined bit in the bitstream is stored in units of bits, and/or an integer multiple n of bits, with $n > 1$, and $n = 8$, for example.

(56) According to an embodiment, a bit position in the bitstream of the previous entry point is signaled in the bitstream.

(57) According to an embodiment, the bitstream pointer is signaled in the bitstream differentially with respect to a further bit stream pointer which is comprised by the entry point information for pointing to a further predetermined bit in the bitstream, which is up to be read next after a resuming the arithmetic decoding the bitstream from a preceding entry point onwards.

(58) According to an embodiment, the bitstream pointer is signaled in the bitstream using a variable length code.

(59) According to an embodiment, the bitstream pointer is signaled in the bitstream using an Exponential-Golomb code, preferably an unsigned Exponential-Golomb code.

(60) According to an embodiment, the predetermined entry point is a third or any of entry points following after the third entry point relative to the begin of the bitstream, and the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of a difference between the offset relative to the previous entry point and an offset of the previous entry point relative to a further entry point preceding the previous entry point.

(61) According to an embodiment, the bitstream pointer is signaled in the bitstream using a signed Exponential-Golomb code.

(62) According to an embodiment, the bit position in the bitstream of the previous entry point is signaled in the bitstream using an unsigned Exponential-Golomb code.

(63) According to an embodiment, an exponential Golomb code parameter for the Exponential-Golomb code is a value of 11.

(64) According to an embodiment, the bitstream pointer is signaled in the bitstream using a signed

Exponential-Golomb code.

(65) According to an embodiment, an exponential Golomb code parameter for the Exponential-Golomb code is a value of 7.

(66) According to an embodiment, the arithmetic decoder is configured to use context-adaptive arithmetic decoding for arithmetically decoding the sequence of symbols, including selecting, for a context-adaptively decoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols, wherein the entry point information is indicative of, for each of a set of one or more predetermined context models, a predetermined probability estimate for the respective predetermined context model, and the arithmetic decoder is configured to use the predetermined probability estimate in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model.

(67) According to an embodiment, the arithmetic decoder is configured to use context-adaptive arithmetic decoding for arithmetically decoding the sequence of symbols, including selecting, for a context-adaptively decoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols, wherein the arithmetic decoder is configured to, for each of a set of one or more predetermined context models, set the probability estimate for the respective predetermined context model at the predetermined entry point to a default state and the arithmetic decoder is configured to use the default state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model.

(68) According to an embodiment, the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward and the arithmetic decoder is configured to for each of the set of one or more predetermined context models, set the probability estimate for the respective predetermined context model at a subsequent predetermined entry point to a saved state manifesting itself at a predetermined condition during arithmetically decoding the sequence of symbols ahead the subsequent predetermined entry point and the arithmetic decoder is configured to use the saved state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model from the subsequent predetermined entry point onwards, e.g. here the decoder might arithmetically decode the bitstream in parallel: once from the beginning using signaled or default context states, and starting from the entry point onwards as soon as the predetermined condition is met, such as a certain number of information values as been decoded or the like.

(69) According to an embodiment, the arithmetic decoder is configured to use dependent dequantization for deriving from the sequence of information values a sequence of quantized values using a state machine, derive from the entry point information a quantization state starting with which the dependent quantization is resumed from the predetermined entry point onwards.

(70) According to an embodiment, the symbols are bins and the desymbolization is a debinarization.

(71) According to an embodiment, the information values is a sequence of syntax elements which represent a video.

(72) According to an embodiment, the information values are neural network parameters.

(73) An embodiment according to the invention creates a method of arithmetically encoding a sequence of information values into an arithmetic coded bitstream, comprising: symbolizing the information values into symbol strings so as to obtain a sequence of symbols; arithmetically encoding the sequence of symbols by subdividing a current interval, which defines a current

version of an coding state of an arithmetic encoder, for each symbol, according to a probability estimate for the respective symbol, selecting a subinterval out of a plurality of the subintervals according to a symbol value of the respective symbol so as to obtain an updated version of the coding state of the arithmetic encoder, defined by the selected subinterval, for encoding a next symbol of the sequence of symbols, and renormalizing encoder-internal parameters which define the coding state under continuing the bitstream, providing the bitstream with entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward.

(74) The method according to this embodiment is based on the same considerations as an apparatus for encoding described above. Moreover, this disclosed embodiment may optionally be supplemented by any other features, functionalities and details disclosed herein in connection with the apparatus for encoding, both individually and taken in combination.

(75) An embodiment according to the invention creates a method of arithmetically decoding a sequence of information values from a bitstream, comprising: deriving from the bitstream entry point information; using the entry point information so as to resume arithmetically decoding the bitstream from a predetermined entry point onward by arithmetically decoding a sequence of symbols from the bitstream by, for each symbol of the bitstream: determining, a current version of an coding state of an arithmetic decoder, a subinterval out of a plurality of subintervals into which a current interval is subdivided according to a probability estimate for the respective symbol, and deducing, based on the selected subinterval, a symbol value of the respective symbol, and renormalizing and updating decoder-internal parameters which define the coding state by use of the bitstream and the selected subinterval so as to obtain an updated version of the coding state of the arithmetic decoder for decoding a next symbol of the sequence of symbols, and deriving the information values from the sequence of symbols by desymbolization.

(76) The method according to this embodiment is based on the same considerations as an apparatus for decoding described above. Moreover, this disclosed embodiment may optionally be supplemented by any other features, functionalities and details disclosed herein in connection with the apparatus for decoding, both individually and taken in combination.

(77) An embodiment according to the invention creates a computer program having a program code for performing, when running on a computer, methods according to any of embodiments described above.

(78) An embodiment according to the invention creates a bitstream generated using an arithmetic encoder of any of the embodiments described herein.

(79) An embodiment according to the invention creates an arithmetic decoder for arithmetically decoding neural network parameters from a bitstream, configured to: arithmetically decode a sequence of symbols from the bitstream by use of context-adaptive arithmetic decoding, including selecting, for a context-adaptively decoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith and arithmetically decoding the context-adaptively decoded symbol using the selected context model, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols, derive the neural network parameters from the sequence of symbols by desymbolization, at the beginning of the bitstreams and/or at one or more entry points within the bitstream, initialize, for each of a set, maybe all, but maybe only a portion of the plurality, of one or more context models, the probability estimate associated with the respective context model based on context model information in the bitstream.

(80) According to an embodiment, the arithmetic decoder is configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols by, for each context model, generating the probability estimate associated with the respective context model on the basis of previously context-adaptively decoded symbols of the sequence of symbols for which the respective context model has been selected.

(81) According to an embodiment, the arithmetic decoder is configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols by deriving a first hypothesis for the probability estimates at a manner which adapts to the actual symbol statistics at a first adaptation agility which is controllable by a first agility parameter, at the beginning of the bitstream and/or at one or more entry points within the bitstream, set, for each of the set of one or more context models, the first hypothesis and the first agility parameter associated with the respective context model based on the context model information in the bitstream.

(82) According to an embodiment, the arithmetic decoder is configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols by deriving a second hypothesis for the probability estimates at a manner which adapts to the actual symbol statistics at a second adaptation agility which is controllable by a second agility parameter, wherein the probability estimates are determined by an average of the first and second hypotheses, at the beginning of the bitstream and/or at one or more entry points within the bitstream, set, for each of the set of one or more context models, the second hypothesis and the second agility parameter associated with the respective context model based on the context model information in the bitstream.

(83) According to an embodiment, the context model information in the bitstream comprises a table entry index into a table of quadruples of values for defining the first and second hypotheses and the first and second agility parameters, and the arithmetic decoder is configured to use the table entry index so as to select one quadruple of the table and to use the one quadruple so as to set the first and second hypotheses and the first and second agility parameters.

(84) According to an embodiment, a number of the quadruples of values for defining the first and second hypotheses and the first and second agility parameters lies between 8 and 10, both inclusively. See the table below where nine lines/quadruples exist.

(85) According to an embodiment, the quadruples of values for defining the first and second hypotheses and the first and second agility parameters correspond to one of 3, 4 or 5 mutually distinguishable settings for the first and second agility parameters. See the table below where (1,4), (2,6), (0,5) or (3,5) exist.

(86) According to an embodiment, the symbols are bins and the desymbolization is a debinarization and the quadruples of values for defining the first and second hypotheses and the first and second agility parameters comprise first 3 quadruples (e.g., cp. 1,4,0,0, 1,4,95,1519 and 1,4,-41,-654) according to all of which the first agility parameter is set to a first value, the second agility parameter is set to a second value, which corresponds to lower adaptation agility than the first value, and according to a first one of which the first and second hypotheses correspond to equiprobability, according to a second one of which the first and second hypotheses correspond to a first bin value being more probable than a second bin value, and according to a third one of which the first and second hypotheses correspond to the second bin value being more probable than the first bin value, second 3 quadruples (e.g., cp. 2,6,95,1519, 2,6,30,482 and 2,6,-21,-337) according to all of which the first agility parameter is set to a third value, which corresponds to lower adaptation agility than the first value, and larger adaptation agility than the second value, and the second agility parameter is set to a fourth value, which corresponds to lower adaptation agility than the second value, and according to a first and second one of which the first and second hypotheses correspond to the first bin value being more probable than the second bin value, and according to a third one of which the first and second hypotheses correspond to the second bin value being more probable than the first bin value, 2 quadruples (e.g., cp. 3,5,0,0 and 3,5,30,482) according to all of which the first agility parameter is set to a fourth value, which corresponds to lower adaptation agility than the third value, and larger adaptation agility than the second value, and the second agility parameter is set to a sixth value, which corresponds to lower adaptation agility than the second value, and larger adaptation agility than the fourth value, and according to a first one of

which the first and second hypotheses correspond to equi-probability, and according to a second one of which the first and second hypotheses correspond to the first bin value being more probable than the second bin value, 1 quadruple (e.g., cp. 0,5,0,0) according to which the first agility parameter is set to a seventh value, which corresponds to larger adaptation agility than the first value, and according to which the first and second hypotheses correspond to equi-probability. (87) According to an embodiment, the symbols are bins and the desymbolization is a debinarization and the arithmetic decoder is configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols by, for each of the first and second hypotheses, representing the respective hypothesis by means of a signed integer, respectively, which, when being zero, is indicative of equi-probability, which, when being larger than zero, is indicative of a first bin value being more probable than a second bin value, and which, when being lower than zero, is indicative of the second bin value being more probable than the first bin value, for each context model, increase the signed integer if a currently decoded bin has the first bin value, and decrease the signed integer if a currently decoded bin has the second bin value, at an amount which is controlled by the first agility parameter with respect to the first hypothesis and controlled by the second agility parameter with respect to the second hypothesis so that the amount is the larger the smaller the first and second agility parameter is, respectively, wherein the probability estimates are determined by an average of the first and second signed integers.

(88) According to an embodiment, the arithmetic decoder is configured to determine the amount for the increase and decrease by use of a transition table.

(89) According to an embodiment, the arithmetic decoder is configured to use the same transition table for the first and second hypotheses.

(90) According to an embodiment, the arithmetic decoder is configured to determine the amount for the increase and decrease by use of the transition table by looking up the transition table at an entry indexed by a table index determined by the signed integer to obtain a transition step and dividing the transition step size by 2 to a power which depends on the first and second adaptation parameter, respectively, wherein the transition step determines the amount.

(91) According to an embodiment, the signed integer is represented by a 2's-complement representation with n bits, with n being larger for the second hypothesis than for the first hypothesis, (8 and 12 in the example below in the detailed description); that is, the "scale" for second hypothesis is larger/finer as compared to the first hypothesis so that the agility is reduced if the transition steps were the same, wherein the arithmetic decoder is configured to determine the amount for the increase and decrease by use of the transition table by looking up the transition table at an entry indexed by a sum of the signed integer divided by $2^{\sup.n-m}$ on the one hand and $2^{\sup.m-1}$ on the other hand ($m=5$ in the example below in the detailed description) in case to obtain a transition step dividing (thus, the adaptation steps which determine adaptation agility are determined by agility parameter) the transition step size by 2 to a power which affine linearly ($4+\text{shift0}$ and shift1 , respectively) depends on the first and second adaptation parameter, respectively, wherein the transition step determines the amount. The dependency may be such that the agility was the same between first and second hypotheses if first and second agility parameter were the same; above this is done by "4+", i.e. by adding the difference of n between first and second hypotheses to the agility parameter of the first hypothesis.

(92) According to an embodiment, the transition steps are stored in the entries of the transition table increase or decrease monotonically.

(93) An embodiment according to the invention creates an arithmetic encoder for arithmetically encoding neural network parameters into a bitstream, configured to: derive from the neural network parameters a sequence of symbols by symbolization, arithmetically encode the sequence of symbols into the bitstream by use of context-adaptive arithmetic encoding, including selecting, for a context-adaptively encoded symbol of the sequence of symbols, a context model out of a plurality

of context models, each of which has a probability estimate associated therewith and arithmetically encoding the context-adaptively encoded symbol using the selected context model, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols, at the beginning of the bitstreams and/or at one or more entry points within the bitstream, initialize, for each of a set of one or more context models, the probability estimate associated with the respective context model according to context model information signaled in the bitstream.

(94) According to an embodiment, the arithmetic encoder is configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols by for each context model, generating the probability estimate associated with the respective context model on the basis of previously context-adaptively encoded symbols of the sequence of symbols for which the respective context model has been selected.

(95) According to an embodiment, the arithmetic encoder is configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols by deriving a first hypothesis for the probability estimates at a manner which adapts to the actual symbol statistics at a first adaptation agility which is controllable by a first agility parameter, at the beginning of the bitstreams and/or at one or more entry points within the bitstream, set, for each of the set of one or more context models, the first hypothesis and the first agility parameter associated with the respective context model according to the context model information signaled in the bitstream.

(96) According to an embodiment, the arithmetic encoder is configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols by deriving a second hypothesis for the probability estimates at a manner which adapts to the actual symbol statistics at a second adaptation agility which is controllable by a second agility parameter and forming an average of the first and second hypothesis, at the beginning of the bitstream and/or at one or more entry points within the bitstream, set, for each of the set of one or more context models, the second hypothesis and the second agility parameter associated with the respective context model according to the context model information signaled in the bitstream.

(97) According to an embodiment, wherein the context model information in the bitstream comprises a table entry index into a table of quadruples of values for defining the first and second hypotheses and the first and second agility parameters, and the arithmetic encoder is configured to use the table entry index so as to select one quadruple of the table and to use the one quadruple so as to set the first and second hypotheses and the first and second agility parameters.

(98) According to an embodiment, a number of the quadruples of values for defining the first and second hypotheses and the first and second agility parameters lies between 8 and 10, both inclusively. See the table below where nine lines/quadruples exist.

(99) According to an embodiment, the quadruples of values for defining the first and second hypotheses and the first and second agility parameters correspond to one of 3, 4 or 5 mutually distinguishable settings for the first and second agility parameters. See the table below where (1,4), (2,6), (0,5) or (3,5) exist.

(100) According to an embodiment, the quadruples of values for defining the first and second hypotheses and the first and second agility parameters comprise first 3 quadruples (e.g., cp. 1,4,0,0, 1,4,95,1519 and 1,4,-41,-654) according to all of which the first agility parameter is set to a first value, the second agility parameter is set to a second value, which corresponds to lower adaptation agility than the first value, and according to a first one of which the first and second hypotheses correspond to equi-probability, according to a second one of which the first and second hypotheses correspond to a first bin value being more probable than a second bin value, and according to a third one of which the first and second hypotheses correspond to the second bin value being more probable than the first bin value, second 3 quadruples (e.g., cp. 2,6,95,1519, 2,6,30,482 and

2,6,-21,-337) according to all of which the first agility parameter is set to a third value, which corresponds to lower adaptation agility than the first value, and larger adaptation agility than the second value, and the second agility parameter is set to a fourth value, which corresponds to lower adaptation agility than the second value, and according to a first and second one of which the first and second hypotheses correspond to the first bin value being more probable than the second bin value, and according to a third one of which the first and second hypotheses correspond to the second bin value being more probable than the first bin value, 2 quadruples (e.g., cp. 3,5,0,0 and 3,5,30,482) according to all of which the first agility parameter is set to a fourth value, which corresponds to lower adaptation agility than the third value, and larger adaptation agility than the second value, and the second agility parameter is set to a sixth value, which corresponds to lower adaptation agility than the second value, and larger adaptation agility than the fourth value, and according to a first one of which the first and second hypotheses correspond to equi-probability, and according to a second one of which the first and second hypotheses correspond to the first bin value being more probable than the second bin value, 1 quadruple (e.g., cp. 0,5,0,0) according to which the first agility parameter is set to a seventh value, which corresponds to larger adaptation agility than the first value, and according to which the first and second hypotheses correspond to equi-probability.

(101) According to an embodiment, the symbols are bins and the symbolization is a binarization and the arithmetic encoder is configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols by, for each of the first and second hypotheses, representing the respective hypothesis by means of a signed integer, respectively, which, when being zero, is indicative of equi-probability, which, when being larger than zero, is indicative of a first bin value being more probable than a second bin value, and which, when being lower than zero, is indicative of the second bin value being more probable than the first bin value, for each context model, increase the signed integer if a currently encoded bin has the first bin value, and decrease the signed integer if a currently encoded bin has the second bin value, at an amount which is controlled by the first agility parameter with respect to the first hypothesis and controlled by the second agility parameter with respect to the second hypothesis so that the amount is the larger the smaller the first and second agility parameter is, respectively, wherein the probability estimates are determined by an average of the first and second signed integers.

(102) According to an embodiment, the arithmetic encoder is configured to determine the amount for the increase and decrease by use of a transition table.

(103) According to an embodiment, the arithmetic encoder is configured to use the same transition table for the first and second hypotheses.

(104) According to an embodiment, the arithmetic encoder is configured to determine the amount for the increase and decrease by use of the transition table by looking up the transition table at an entry indexed by a table index determined by the signed integer to obtain a transition step and dividing the transition step size by 2 to a power which depends on the first and second adaptation parameter, respectively, wherein the transition step determines the amount.

(105) According to an embodiment, the signed integer is represented by a 2's-complement representation with n bits, with n being larger for the second hypothesis than for the first hypothesis (8 and 12 in the example below in the detailed description; that is, the "scale" for second hypothesis is larger/finer as compared to the first hypothesis so that the agility is reduced if the transition steps were the same), wherein the arithmetic encoder is configured to determine the amount for the increase and decrease by use of the transition table by looking up the transition table at an entry indexed by a sum of the signed integer divided by $2^{\sup.n-m}$ on the one hand and $2^{\sup.m-1}$ on the other hand ($m=5$ in the example below in the detailed description) in case to obtain a transition step dividing (thus, the adaptation steps which determine adaptation agility are determined by agility parameter) the transition step size by 2 to a power which affine linearly

(4+shift0 and shift1, respectively) depends on the first and second adaptation parameter, respectively, wherein the transition step determines the amount. The dependency may be such that the agility was the same between first and second hypotheses if first and second agility parameter were the same; above this is done by “4+”, i.e. by adding the difference of n between first and second hypotheses to the agility parameter of the first hypothesis.

(106) According to an embodiment, the transition steps are stored in the entries of the transition table increase or decrease monotonically.

(107) The arithmetic encoder, the arithmetic decoder, the method of arithmetically encoding, the method for arithmetically decoding, the computer program for implementing these methods, the arithmetic encoder for arithmetically encoding neural network parameters and a bitstream may optionally be supplemented by any of the features, functionalities and details disclosed herein (in the entire document), both individually and taken in combination.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) Embodiments of the present invention will be detailed subsequently referring to the appended drawings, in which:

(2) FIG. 1 shows a flow chart of a method **100** for encoding in accordance with an embodiment;

(3) FIG. 2 shows a flow chart of a method **200** for decoding in accordance with an embodiment;

(4) FIG. 3 shows a schematic representation of encoding and decoding parameters occurring during encoding and decoding an arithmetic bitstream with illustrating different possibilities as to what may be signaled as entry point information in the bitstream in accordance with embodiments.

DETAILED DESCRIPTION OF THE INVENTION

(5) A method **100** of arithmetically encoding a sequence of integer values into an arithmetic coded bitstream in accordance with an embodiment is presented in FIG. 1.

(6) The method comprises symbolizing at step **101** the information values into symbol strings so as to obtain a sequence of symbols and arithmetically encoding at step **102** the sequence of symbols.

(7) The arithmetically encoding comprises subdividing at step **103** a current interval, which defines a current version of an coding state of an arithmetic encoder, for each symbol, according to a probability estimate for the respective symbol. The arithmetically encoding further comprises selecting at step **104** a subinterval out of a plurality of the subintervals according to a symbol value of the respective symbol. An updated version of the coding state of the arithmetic encoder, defined by the selected subinterval, is thus obtained, which is further used for encoding a next symbol of the sequence of symbols. The arithmetically encoding further comprises renormalizing at step **105** encoder-internal parameters which define the coding state under continuing the bitstream.

(8) After performing the arithmetically encoding the method **100** further provides at step **106** the bitstream with entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward.

(9) However, it should be noted that the method **100** may optionally be supplemented by any of the features, functionalities and details disclosed herein, both individually or taken in combination.

(10) A method **200** of arithmetically decoding a sequence of information values from a bitstream in accordance with an embodiment is presented in FIG. 2.

(11) The method comprises deriving at step **201** entry point information from the bitstream and using at step **202** the entry point information so as to resume arithmetically decoding the bitstream from a predetermined entry point onward by arithmetically decoding a sequence of symbols from the bitstream, for each symbol of the bitstream. The arithmetically decoding the sequence of symbols from the bitstream comprises determining at step **203**, a current version of an coding state of an arithmetic decoder and a subinterval out of a plurality of subintervals into which a current

interval is subdivided according to a probability estimate for the respective symbol. The arithmetically decoding further comprises deducing at step **204**, based on the selected subinterval, a symbol value of the respective symbol. The arithmetically decoding further comprises renormalizing and updating at step **205** decoder-internal parameters which define the coding state by use of the bitstream and the selected subinterval so as to obtain an updated version of the coding state of the arithmetic decoder for decoding a next symbol of the sequence of symbols.

(12) After performing the arithmetically decoding the method **200** further derives at step **206** the information values from the sequence of symbols by desymbolization.

(13) However, it should be noted that the method **200** may optionally be supplemented by any of the features, functionalities and details disclosed herein, both individually or taken in combination.

(14) FIG. 3 is a schematic representation of encoding and decoding parameters as developing during encoding and decoding, with illustrating as to what could be signaled as entry point information in the bitstream to be used in the decoding process.

(15) At the beginning of the encoding and decoding at the begin of the bitstream or at a leading end from which onwards the bitstream is to be decoded completely the following variables might be initialized in a default manner: interval width R both at encoder and decoder, interval offset L at the encoder, a state of context model (optionally if context adaptivity applies) both at encoder and decoder and, optionally, a quantization state (if dependent quantization is used). The pointer V might be going to be derived by the decoder from the first few bits in the bitstream. From the beginning inwards, the parameters develop and a decoder would have to perform the whole decoding process to know as to which states occur at any entry point. According to embodiments, the encoder assumes responsibility to generate this information and convey such information along with the bitstream to the decoder so that the latter might directly commence decoding at one of the entry points.

(16) Each of the entry points of the bitstream could be seen as a snapshot of the state of the arithmetic encoder or, to be more precise, of an arithmetic decoding process run in the encoder to simulate that state at encoder site. Particularly, the entry points of the bitstream are quasi indicative of a snapshot of states of context models or a quantization state of the decoding process as occurring during decoding the bitstream in question. The indicated variables are thus signaled in the bitstream for the predetermined entry points. They might be signalled in a header preceding the bitstream, following the bitstream or inserted inbetween such as at the entry points themselves. Parts of the parameters needed for decoding might be set at encoder and decoder in a synchronous manner at the entry points, so that no information thereabout needs to be included in the entry point information.

(17) In the following, an embodiment is described for arithmetically encoding a sequence of information values into an arithmetic coded bitstream, e.g. in a bitstream containing encoded neural network data, e.g. in a NNR bitstream. Entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward is shown in Table 1 in form of a pseudo code. Naturally, the details set out herein also reveal a corresponding arithmetic decoding of the thus generated bitstream.

(18) TABLE-US-00001 TABLE 1 for $j=0; j < \text{NumBlockRowsMinus1}; j++$) { cabac_offset_list[j] u(8) if(dq_flag) dq_state_list[j] u(3) if($j == 0$) { bit_offset_delta1 ue(11) BitOffsetList[j] = bit_offset_delta1 } else { bit_offset_delta2 ie(7) BitOffsetList[j] = BitOffsetList[$j-1$] + bit_offset_delta2 } }

(19) According to FIG. 1, the information values are symbolized into symbol strings so as to obtain a sequence of symbols, which are arithmetically encoded by, for each symbol, selecting a subinterval out of a plurality of subintervals into which a current interval, which defines a current version of an coding state of the arithmetic encoder, is subdivided according to a probability estimate for the respective symbol, the selection being done according to a symbol value of the respective symbol, so as to obtain an updated version of the coding state of the arithmetic encoder,

defined by the selected subinterval, for encoding a next symbol of the sequence of symbols, and renormalizing encoder-internal parameters which define the coding state under continuing the bitstream.

(20) The bitstream is provided with entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward.

(21) As could be seen in Table 1, the entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward includes: `cabac_offset_list`, which specifies a list of values to be used to initialize variable `IvlOffset` at the beginning of entry points. It is exemplarily shown to be coded as a 8-bit unsigned integer, but it could also be coded differently; in this example, variables `IvlCurrRange` and `IvlOffset` are used to define the status of the arithmetic decoding engine, or the coding state; that is, the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval; here, `IvlCurrRange` denotes the width of the interval and `IvlOffset` indicates the pointer into the interval. `dq_state_list` specifies a list of values to be used to initialize variable `stateId` at the beginning of entry points. It is exemplarily shown to be coded as a 3-bit unsigned integer, but it could also be coded differently; `stateId` is used here as a variable denoting the state of the state machine used to perform dependent quantization. `bit_offset_delta1` specifies the first element of list `BitOffsetList`. It is exemplarily shown to be coded using an unsigned 11-th order Exp-Golomb code, but it could also be coded differently; `bit_offset_delta2` specifies elements of list `BitOffsetList` except for the first element, as difference to the previous element of list `BitOffsetList`. It is exemplarily shown to be coded using a signed 7-th order Exp-Golomb code;

(22) Variable `BitOffsetList` is a list of bit offsets to be used to set the bitstream pointer position at the beginning of entry points.

(23) To be more precise, Table 1 relates to an embodiment where dependent quantization is used for deriving the sequence of information values from a sequence of unquantized values, namely using a state machine, wherein the entry point information comprises a quantization state manifesting itself in the state machine up to the predetermined entry point, namely `stateId`.

(24) The manner at which the arithmetic decoding the bitstream from a predetermined entry point onward is launched using the entry point information in Table 1 is exemplarily shown in Table 2 in form of a pseudo code.

(25) TABLE-US-00002 TABLE 2

```
if( entryPointOffset = -1 &&      GetEntryPointIdx(
dimensions, i, scan_order != -1 &&      scan_order > 0 ) {   IvlCurrRange = 256   j =
entryPointOffset +      GetEntryPointIdx( dimensions, i, scan_order )   IvlOffset =
cabac_offset_list[ j ]   if ( dq_flag )      stateId = dq_state_list[ j ]   set_bit_pointer( bitpointer +
lastOffset + BitOffsetList[ j ] )   lastOffset = BitOffsetList[ j ]   init_prob_est_param( ) }
```

(26) In Table 2, a variable `entryPointOffset` is used to indicate whether entry points are present for decoding and, if entry points are present, an entry point offset.

`GetEntryPointIdx(tensorDimensions[ ], i, scan)` returns `-1` if index `i` doesn't point to the first position of an entry point. If index `i` points to the first position of an entry point, it returns the entry point index within the tensor.

(27) `GetEntryPointIdx(tensorDimensions[ ], i, scan)` returns `-1` if index `i` doesn't point to the first position of an entry point. If index `i` points to the first position of an entry point, it returns the entry point index within the tensor. To determine the positions and indexes of entry points, the following applies: A variable `w` is set to `Prod(tensorDimensions)/tensorDimensions[0]` A variable `epIdx` is set to `i/(w*(4<<scan))-1` If `i>0` and `i % (w*(4<<scan))` is equal to 0, index `i` points to the first position of an entry point and the entry point index is equal to `epIdx`.

(28) Otherwise, index `i` doesn't point to the first position of an entry point. `dq_flag` specifies whether the quantization method is dependent scalar quantization or uniform quantization. A `dq_flag` equal to 0 indicates that the uniform quantization method is used. A `dq_flag` equal to 1

indicates that the dependent scalar quantization method is used. If `dq_flag` is not present, it is inferred to be 0. `set_bit_pointer` sets the position of the bitstream pointer. `init_prob_est_param()` invokes the initialization process for probability estimation parameters. `scan_order` specifies the block scanning order for parameters with more than one dimension according to the following table: 0: No block scanning 1: 8×8 blocks 2: 16×16 blocks 3: 32×32 blocks 4: 64×64 blocks (29) That is, in the example provided by Table 1 and 2, the entry point information comprises a bitstream pointer to a predetermined bit in the bitstream, which is up to be read next after the resuming the arithmetic decoding the bitstream from the predetermined entry point onwards, namely `set_bit_pointer`. Here, the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward, namely for `NumBlockRowsMinus1` entry points. Entry points might be defined to be positioned between row changes when scanning a tensor row wise or regularly by scanning one dimension completely before scanning the next row along that dimension and so forth. The bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a previous entry point as derivable from the addition in `BitOffsetList[j]=BitOffsetList[j-1]+bit_offset_delta2` in Table 1.

(30) At each entry point `j`, the coding state is initialized using `cabac_offset_list[j]` and `dq_state_list[j]` with respect to interval pointer and dependent quantization state, respectively. In accordance with this embodiment here, the interval width is set to a constant or predefined fixed value at each entry point, i.e. an default value, namely here exemplarily 256, wherein `IvlCurrRange` and `IvlOffset` are both in 16 bit register precision in this example. The latter explicit bit precisions are, however, merely examples and could be varied.

Further Embodiments and Aspects

(31) In the following, further aspects and embodiments according to the invention will be described, which can be used individually or in combination with any other embodiments disclosed herein.

(32) Moreover, the embodiments disclosed in this section may optionally be supplemented by any other features, functionalities and details disclosed herein, both individually and taken in combination.

(33) Entry point structures for arithmetic coding in accordance with an embodiment of the invention are further described.

Background

(34) Context-based adaptive binary arithmetic coding (CABAC) is a method for encoding and decoding a sequence of symbols. A binarization stage converts each symbol of such a sequence into a sequence of one or more binary symbols (bins) and the concatenation of these bin sequences is arithmetically encoded into a bitstream. A context modeling stage associates a probability estimate with each bin for arithmetic coding based on previously encoded bins and context information. The decoder has the same information available and can reproduce the same probability estimates in order to carry out arithmetic decoding.

(35) Review of the Binarization Stage and the Context Modeling Stage of CABAC

(36) Binarization and context modeling strongly depends on the application. For example, the video compression standards H.265/HEVC and H.266/NVC both employ CABAC as arithmetic coding engine but have very different binarization and context modeling stages as they are tuned to the types of syntax elements that occur. However, most applications that employ CABAC maintain a set of so-called context models and binarization and context modeling corresponds to associating each bin unambiguously to a particular context model from this set. A context model is usually implemented as backward-adaptive probability estimator that only takes bins into account that were previously associated with the context model.

(37) In the following, three examples for different concepts of implementing context models in accordance with an embodiment of the invention are reviewed:

(38) TABLE-US-00003 Identifier HEVC VVC NNR Notes Used in the Used in the

H.265/HEVC video H.266/VVC video committee draft of compression the ISO/IEC 15938 standard standard Part 17 standard for compression of neural networks (this document contains some incorrect equations, see note below the table) State variables A 7 bit unsigned A 10 bit and a 14 bit An 8 bit and a 12 bit integer unsigned integer signed integer State to probability logarithmic linear Logarithmic mapping Number of — 2 2 adaptation rate variables Details about the implementations can be found in the respective resources in row “Notes”. Note: The committee draft of the ISO/IEC 15938 Part 17 standard contains some incorrect equations in the description of the probability estimator, which need to be corrected as follows in order to yield the correct “NNR” context model implementation: In section 11.3.4.3.2.1, equation “valMps = pStateIdx0 + pStateIdx0 >= 0” has to be replaced with “valMps = 16 * pStateIdx0 + pStateIdx1 >= 0”. In section 11.3.4.3.2.1, equation “ivlLpsRange = rps_table[(abs(pStateIdx0 + pStateIdx1) >> 7)) + qRangeIdx]” has to be replaced with “ivlLpsRange = rps_table[(abs(16 * pStateIdx0 + pStateIdx1) >> 7)) + qRangeIdx]”. In section 11.3.4.3.2.2, equation “pStateIdx0 += sign * (transition_table[16 + (sign * pStateIdx0 >> 3)] >> shift0)” has to be replaced with “pStateIdx0 += sign * (transition_table[16 + (sign * pStateIdx0 >> 3)] >> (4 + shift0))”. In section 11.3.4.3.2.2, equation “pStateIdx1 += sign * (transition_table[16 + (sign * pStateIdx0 >> 7)] >> shift1)” has to be replaced with “pStateIdx1 += sign * (transition_table[16 + (sign * pStateIdx1 >> 7)] >> shift1)”. (39) According to an embodiment of the invention, only the variables that represent the state of a context model are of interest as the probability estimation processes are not changed. Note that the present embodiments of the invention can also be applied to other types of probability estimators that are not described here.

(40) Furthermore, in embodiments of the invention there may exist bins that are not associated with context models like the well-known bypass bins or terminating bins.

(41) Detailed review of the probability estimator in accordance with an embodiment of the invention of the committee draft of the ISO/IEC 15938 Part 17 standard.

(42) Each context model maintains, for example, four variables shift0, shift1, pStateIdx0, and pStateIdx1. The variables pStateIdx0 and pStateIdx1 are signed 8 and 12 bit integers, respectively, in two's complement representation.

(43) A probability estimate for arithmetic encoding or decoding can be derived, for example, from pStateIdx0 and pStateIdx1 and from the width of the current coding interval IvlCurrRange of the arithmetic encoder or decoder, which is a value in the interval [256, 510], as follows:

valMps=16*pStateIdx0+pStateIdx1>=0

qRangeIdx=ivlCurrRange&0xe0 rpls_table=[128, 112, 97, 84, 74, 65, 57, 50, 45, 39, 34, 30, 27, 23, 20, 18, 15, 14, 12, 11, 10, 9, 7, 7, 5, 5, 4, 4, 3, 3, 2, 2, 142, 125, 108, 93, 82, 72, 63, 56, 50, 43, 38, 33, 30, 26, 22, 20, 17, 16, 13, 12, 11, 10, 8, 8, 6, 6, 5, 5, 3, 3, 2, 2, 156, 137, 119, 103, 90, 79, 70, 61, 55, 48, 42, 37, 33, 28, 24, 22, 19, 17, 15, 13, 12, 11, 9, 9, 6, 6, 5, 5, 4, 4, 2, 2, 171, 150, 130, 112, 99, 87, 76, 67, 60, 52, 46, 40, 36, 31, 27, 24, 21, 19, 16, 15, 13, 12, 10, 10, 7, 7, 6, 6, 4, 4, 3, 3, 185, 162, 141, 121, 107, 94, 82, 73, 65, 56, 50, 43, 39, 34, 29, 26, 22, 21, 17, 16, 14, 13, 11, 11, 8, 8, 6, 6, 4, 4, 3, 3, 199, 175, 152, 131, 115, 101, 89, 78, 70, 61, 54, 47, 42, 36, 31, 28, 24, 22, 19, 17, 15, 14, 12, 12, 8, 8, 7, 7, 5, 5, 3, 3, 213, 187, 163, 140, 123, 108, 95, 84, 75, 65, 58, 50, 45, 39, 33, 30, 26, 24, 20, 18, 16, 15, 13, 13, 9, 9, 7, 7, 5, 5, 3, 3, 228, 200, 174, 150, 132, 116, 102, 90, 80, 70, 62, 54, 48, 42, 36, 32, 28, 26, 22, 20, 18, 16, 14, 14, 10, 10, 8, 8, 6, 6, 4, 4]

ivlLpsRange=rpls_table[(abs((16*pStateIdx0+pStateIdx1)>>7))+qRangeIdx]

(44) Variable valMps is the value of the more probable symbol (MPS) and the probability estimate of the less probable symbol (LPS) is given as p.sub.LPS=ivlLpsRange/ivlCurrRange. Accordingly, the probability estimate of the more probable symbol is simply p.sub.MPS=1-p.sub.LPS.

(45) The probability Pr(bin==1) of the next bin being equal to 1 can be derived, for example, as follows: If valMps==1: Pr(bin==1)=p.sub.MPS Otherwise (if valMps==0): Pr(bin==1)=p.sub.LPS

(46) Note that these probability estimates don't directly occur in the encoder or decoder as only the subinterval widths associated with the MPS and LPS are needed for arithmetic encoding or

decoding.

(47) At the beginning of the encoding, pStateIdx0 and pStateIdx1 are both set to 0 which approximately corresponds to p.sub.LPS=0.5.

(48) After encoding or decoding of a bin with value binVal, pStateIdx0 and pStateIdx1 are updated, for example, according to the following equations using variables shift0 and shift1:

transition_table=[2512, 2288, 2064, 1840, 1616, 1392, 1168, 944, 720, 560, 464, 368, 272, 208, 144, 80, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64, 0]

sign=2*binVal-1

pStateIdx0+=sign*(transition_table[16+(sign*pStateIdx0>>3)]>>(4+shift0))

pStateIdx1+=sign*(transition_table[16+(sign*pStateIdx1>>7)]>>shift1)

(49) As can be seen from the equations, larger values for shift0 or shift 1 lead to a smaller modification of the values of pStateIdx0 or pStateIdx1, respectively, which corresponds to a smaller change in the resulting probability estimate while smaller values correspond to larger changes. Consequently, shift0 and shift1 can be seen as agility parameters that control the update agility of pStateIdx0 and pStateIdx1. Typical values for (shift0, shift1) are, for example, (1, 4), (0, 0), (0, 5), (1, 1), (1, 2), (2, 4), (2, 6), (3, 4), or (3, 5) and it depends on the statistical properties of the sequence of bins to be encoded which values are most appropriate for shift0 and shift1.

(50) The underlying principle is known as exponentially weighted moving averages (EWMA).

(51) Review of the arithmetic coding engine of CABAC (M coder) in accordance with an embodiment of the invention is further described.

(52) The M coder in accordance with an embodiment maintains in the decoder two unsigned B bit integer variables V and R. Typically, B is set to 9 but other choices are also possible. At the beginning of the decoding, V is initialized with the first B bit of the arithmetically encoded bitstream (advancing the bitstream pointer by B bits) and R is set to an initial value like e.g. 2.sup.B-2. Note that the bitstream may contain further bits before arithmetic encoding starts such as high-level syntax. R represents the width of the current coding interval and it may only contain values in the interval [2.sup.B-1, 2.sup.B-1]. The coding interval is given as [0, R] and V represents a pointer into the coding interval, i.e., V<R has to hold. For decoding of a bin b, a probability for the two possible values of b is estimated employing well-known techniques like context modeling and probability estimation like e.g. exponentially weighted moving averages (EWMA). Based on the estimated probability associated with bin b, two subintervals I.sub.L=[0, R.sub.L] and I.sub.R=[R.sub.L, R] are derived where I.sub.L is associated with one possible value of bin b and I.sub.R is associated with the other possible value of bin b. For example, I.sub.R is usually associated with the less probable symbol (LPS) based on the estimated probability while I.sub.L is associated with the more probable symbol (MPS).

(53) Decoding of a bin is carried out, for example, as follows (enumerated lists correspond to ordered steps): If V<R.sub.L: 1. The symbol value associated with I.sub.L is the decoded value of b. 2. R is set to R.sub.L. Otherwise (V>=R.sub.L): 1. The symbol value associated with I.sub.R is the decoded value of b. 2. R is set to R-R.sub.L. 3. V is set to V-R.sub.L. If afterwards R<2.sup.B-1, renormalization is carried out, for example, as follows: 1. R is set to 2*R 2. V is set to 2*V+ReadOneBit() 3. If R<2.sup.B-1, continue with step 1 (otherwise renormalization is done).

(54) Function ReadOneBit() returns the next bit in the bitstream and advances the bitstream pointer by 1 bit.

(55) The above bin decoding procedure is also known as regular coding mode. Furthermore, there exists the well-known bypass coding mode and the terminating bin coding mode.

(56) Concept in accordance with an embodiment of the invention is further described.

(57) Arithmetic decoding is a sequential process that can hardly be parallelized. The concept in accordance with an embodiment of the invention introduces a method for enabling a decoder to start decoding at predefined positions in the bitstream by defining so-called entry points. In this way, several decoders can operate in parallel decoding different parts of a single bitstream (starting

at different entry points).

(58) Entry Points

(59) An entry point can be seen as a snapshot of the state of the arithmetic decoder before decoding a particular bin. I.e., it consists of the following variables: a. R b. V c. Pointer to the next bit in the bitstream d. State of the CABAC decoder (e.g. states of context models and potentially further relevant variables like the state of a dependent quantization scheme)

(60) Such an entry point would need $B-1$ bits for R (for representing all possible values in the interval $[2^{\text{sup.B-1}}, 2^{\text{sup.B-1}}]$), B bits for V and further bits for storing c. and d. For storing c., it may be appropriate in an embodiment to employ a variable length code, like an Exponential-Golomb code because small pointer values would result in short binary codewords. Storing d. may, however, need a huge number of bits because there could be a large number of context models and each context model might need several bits to represent its internal state.

(61) Remarkably, an entry point can also be created in an embodiment for existing CABAC bitstreams without a need to re-encode the bitstream.

(62) In an embodiment, one or more entry points have been generated for a CABAC bitstream. For the first entry point, the pointer to the next bit in the bitstream c. is stored as difference between the bit position where arithmetic decoding starts and the bit position of the first entry point.

(63) In another embodiment, this difference is further reduced by B (as the first B bits are loaded into V during the initialization of the arithmetic decoder).

(64) In another embodiment, all entry points after the first entry point (if present) store the pointer to the next bit in the bitstream c. relative to the pointer to the next bit in the bitstream c. of the previous entry point. For example, the difference between the pointers to the next bit in the bitstream c. of the current entry point and the previous entry point is stored.

(65) In another embodiment, the integer value to be stored indicating the pointer to the next bit in the bitstream c. (for example, given as a difference as described in the previous preferred embodiments) is encoded in the bitstream using an unsigned Exponential-Golomb code (which e.g. corresponds to data type $ue(k)$ as defined in the committee draft of ISO/IEC 15938 Part 17) of order k.

(66) In another embodiment, the pointers to the next bit in the bitstream c. involved in the calculation of an integer that represents the next bit in the bitstream c. (as described in the previous preferred embodiments) are rounded down to multiples of 8 before carrying out the calculation. The resulting difference is therefore also a multiple of 8 and can be divided by 8 yielding a byte offset (instead of a bit offset). Furthermore, 3 additional bits are encoded in an entry point, indicating the bit position in the current byte of the current entry point.

(67) In an embodiment, encoding and decoding involves the well-known dependent quantization scheme (like e.g. used in the committee draft of ISO/IEC 15938 Part 17) where a state variable is maintained. A fixed number of bits is stored with each entry point to indicate the value of the dependent quantization state. For example, for the committee draft of ISO/IEC 15938 Part 17, a 3 bit variable is stored indicating one of the 8 possible values of the dependent quantization state.

(68) Double differential signaling of the pointer to the next bit in the bitstream c in accordance with an embodiment is further described.

(69) In an application where the entry points are located roughly equally distant in the bitstream, the differences between pointers to the next bit in the bitstream c. between neighboring entry points have roughly the same value. In such a situation, it may be more efficient to signal “differences of differences” of these bit positions for all except the first two entry points. More precisely, consider three neighboring entry points ep1, ep2, and ep3 with associated pointers to the next bit in the bitstream c. given as bitPos1, bitPos2, and bitPos3, respectively. Furthermore, assume that $\text{bitPos1} < \text{bitPos2} < \text{bitPos3}$ holds. Then, for roughly equally spaced entry points ep1, ep2, and ep3, the differences $d1 = \text{bitPos2} - \text{bitPos1}$ and $d2 = \text{bitPos3} - \text{bitPos2}$ have roughly the same value. I.e., the difference of differences $d2 - d1$ has a small magnitude. Therefore, in an embodiment, the position

of the next bit in the bitstream c. for entry point ep3 is signaled as “double difference” $dd3=d2-d1$. Note that dd3 can now also be negative. As the magnitude of dd3 tends to be smaller than the magnitude of d3, it may be possible to signal it with less bits in the bitstream when, for example, using a signed exponential Golomb code.

(70) In another embodiment, difference d1, indicating the position of the next bit in the bitstream c. of entry point ep2, is signaled as unsigned exponential Golomb code with parameter $k=11$.

(71) In another embodiment, difference dd3, indicating the position of the next bit in the bitstream c. of entry point ep3 (represented as “double difference” as discussed above), is signaled as signed exponential Golomb code with parameter $k=7$.

(72) In another embodiment, differential and/or double differential values (as described above), indicating positions of the next bit in the bit stream c. are calculated based on entry points that are not encoded in the bitstream but, instead, derived by encoder and decoder at predetermined positions. For example, the first entry point ep1 is not signaled in the bitstream (instead, derived in the encoder and decoder), but used to calculate differences or double differences for signaling ep2.

(73) Entry points with encoder modifications in accordance with an embodiment of the invention are further described.

(74) As discussed in the previous section, storing d. in an entry point may need a substantial number of bits. In this section, several concepts are described that reduce the size of an entry point by allowing encoder modifications.

(75) Modifications to the state of the CABAC encoder and decoder (d.) in accordance with embodiments are further described.

(76) In accordance with an embodiment, the state of the CABAC encoder and decoder d. is set to predefined values at the beginning of an entry point. Consequently, it is not necessary to store d. in an entry point. However, this may result in a larger arithmetically encoded bitstream as the probability modeling accuracy is reduced.

(77) For example, the state of the CABAC encoder and decoder at the beginning of an entry point could be set according to one of the following rules (each one in accordance with an embodiment):
1. The state of the CABAC encoder and decoder is reset to default values. 2. The state of the CABAC encoder and decoder is initialized using information that is signaled in the bitstream. 3. The state of the CABAC encoder and decoder is set to a previously saved state of the CABAC encoder and decoder.

(78) Recall that the state of a CABAC encoder and decoder may consist of a set of context models and on further variables. It usually makes sense to apply different rules to different parts of the state of the CABAC encoder and decoder. For example, in accordance with an embodiment, rule 1 could be applied to the context models while applying no rule at all might be more suitable for other variables like the state of a dependent quantization scheme. I.e., the other variables are stored with the entry point. Or in another example, in accordance with an embodiment, rule 1 or rule 2 is applied to a first subset of the context models while rule 2 or rule 3 is applied to a second (disjoint) subset of context models.

(79) Examples based on rule 1 in accordance with an embodiment are: Rule 1 is possibly the simplest method of avoiding the need to store d. with an entry point. However, it may also result in a substantially increased arithmetically coded bitstream as the probability modeling accuracy is strongly reduced.

(80) Examples based on rule 2 in accordance with an embodiment are: Rule 2 can, for example be applied to context models only while other rules (or no rule) can be applied to all other variables (if any) that are associated with the state of the CABAC encoder and decoder (like e.g. the state of a dependent quantization scheme). For example, initialization for some or all context models can be signaled in the bitstream somewhere before the first entry point. At the beginning of each entry point this initialization information is used to derive initial states for each context model.

(81) Examples based on rule 3 in accordance with an embodiment are:

(82) Storing and loading states of a context model is, for example, known from the context of wavefront parallel processing of the video compression standard H.265/HEVC. The encoder and decoder store the states of context models at predefined positions and have them available to load these states at other predefined positions. This concept can be combined with the entry point concept where saved context model states are loaded at the beginning of an entry point.

(83) Modifications to variable R in the CABAC encoder and decoder in accordance with embodiment are further described.

(84) The variable R is in the range $[2^{\text{sup.B}-1}, 2^{\text{sup.B}-1}]$. Consequently, storing R in an entry point needs $B-1$ bit (because there are $2^{\text{sup.B}-1}-2^{\text{sup.B}-1}+1=2^{\text{sup.B}-1}$ values in the interval). Signaling R in an entry point can be avoided in an embodiment by encoding and decoding a so-called pseudo-bin before creating an entry point. This is done by using the encoding procedure for a regular bin with $R_{\text{sub.L}}$ set to $2^{\text{sup.B}-1}$ and with selecting the symbol value associated with $L_{\text{sub.L}}$ as the encoded or decoded symbol. Note that such a pseudo-bin is only needed if $R > 2^{\text{sup.B}-1}$. After such a pseudo-bin, the variable R is $2^{\text{sup.B}-1}$ and therefore, it needs not to be stored in an entry point which saves $B-1$ bits. As the pseudo bin is the more probable symbol (MPS), it produces an average portion of bits in the bit stream that is less than 1 bit which is less than the saved $B-1$ bits. This technique also has an impact on the possible values of variable V since $V < R$ has to hold. Consequently, $V < 2^{\text{sup.B}-1}$ holds as well and V can be signaled in the bitstream with $B-1$ bits instead of B bits saving one more bit in an entry point.

(85) In an embodiment, R is set to $2^{\text{sup.B}-1}$ at the beginning of an entry point (in encoder and decoder) and V is stored as fixed length variable with $B-1$ bit.

(86) Application of entry points to the compression of 2D arrays of values in accordance with an embodiment of the invention is further described.

(87) Several applications encode 2D arrays of values into a bitstream. For example, image or video compression schemes encode pictures that are represented as 2D arrays of samples or compression schemes for neural network parameters reshape parameter tensors into 2D structures in order to encode the contained parameter values. Such schemes usually apply block partitioning techniques on the 2D arrays and they also define an order of the resulting blocks for encoding or decoding. Furthermore, such schemes may also contain lossy operations, like e.g. quantization, and the decoded and reconstructed 2D array is not identical to the original 2D array but usually similar (for example, in a visual sense in the case of video compression).

(88) Consider, for example, a video or neural network parameter compression scheme that subdivides a 2D array (like an image plane or a reshaped parameter tensor) into blocks of size $N \times N$ and defines a scan order in which the blocks are ordered for encoding them into a bitstream. Such a scheme forms rows and columns of $N \times N$ blocks. For example, the order within a row could start with the left-most block and proceeding to the right. The order of the rows could simply start with the uppermost row proceeding downwards. This scan order is also known as raster scan and it could be desirable to enable the decoder to decode all rows in parallel. In this case, one entry point at the beginning of each row would be needed. Note, however, that in an embodiment a decoder might need information of neighboring blocks in order to be able to decode a current block. For example, video compression algorithms usually access information of neighboring blocks (e.g. to the left or to the top) in order to be able to decode the current block. In this case, a decoder has to further ensure that these blocks are decoded before decoding of the current block can be done.

(89) Furthermore, parallel decoding may be useful for some 2D arrays while for others it might not be necessary. For example, for large 2D arrays it may be useful while for small 2D arrays it might not be necessary. In an embodiment, it is signaled in the bitstream whether entry points are present for a 2D array.

(90) In an embodiment, the states of the context models (according to rule 3) are stored after the first block of each row. One entry point is generated at the beginning of each block row where the states of the context models are set to the stored values as present after encoding or decoding of the

first block of the neighboring row above.

(91) In another embodiment, the states of the context models (according to rule 3) are stored after the first block of the first row. One entry point is generated at the beginning of each block row where the states of the context models are set to the same values as present after encoding or decoding of the first block of the uppermost row.

(92) Signaling of shared context model initialization information for entry points in accordance with an embodiment is further described.

(93) Before the beginning of CABAC decoding, the states of all context models has to be set to predefined values. This can, for example, be default values or by using a more sophisticated context model initialization procedure. For example, the committee draft of the ISO/IEC 15938 Part 17 compression standard for neural networks (including the corrected equations as discussed above) employs two state variables per context model (pStateIdx0 and pStateIdx1) and associates a shift parameter (shift0 and shift1) with each of the state variables. The shift parameter controls the adaptation agility of the state variable update that is based on an EWMA estimator. At the beginning of the arithmetic encoding or decoding, the two state variables of each context models are set to 0, which corresponds to an initial probability estimate of 0.5. Furthermore, there exist 9 predefined value pairs for the initialization of the two shift parameters (see array ShiftParameterSet) and it is signaled in the bitstream for each context model which of the 9 value pairs is used for the shift parameters.

(94) The effect of also distinguishing several different initialization values for the state variables of a context model is usually rather limited because the underlying EWMA estimator quickly adapts to the statistics of the bin sequence associated with a context model. However, if entry points are present and if context models are initialized at each entry point, this effect becomes more relevant. Therefore, it may be beneficial to allow a set of different values for the initialization of the state variables of a context model.

(95) In an embodiment, a list of 4-tuples is defined, each 4-tuple representing initial values for the two state variables and the two shift parameters of a context model. In the case of NNR, the values of a 4-tuple would be associated with the 4-tuple of variables (shift0, shift1, pStateIdx0, pStateIdx1). Before arithmetic coding starts, an integer index is signaled in the bitstream for each context model, indicating which of the 4-tuples is used for initializing the respective context model.

(96) In another embodiment, the index indicating the 4-tuple to be used to initialize a context model is signaled using a variable length code that assigns shorter code words to more frequently used 4-tuples.

(97) For example, the following list of 4-tuples (shift0, shift1, pStateIdx0, pStateIdx1) could be used:

(98) TABLE-US-00004

Index	shift0	shift1	pStateIdx0	pStateIdx1
0	1	4	0	1
1	1	4	-41	-654
2	1	4	95	1519
3	0	5	0	4
4	2	6	30	482
5	2	6	95	1519
6	2	6	-21	-337
7	3	5	0	8
8	3	5	30	482

(99) Note that the same concept can also be used with the context models of VVC where the two state variables and the two agility parameters use the same variable names as NNR (shift0, shift1, pStateIdx0, pStateIdx1). Note that these variables fulfill the same purpose in VVC as in NNR although their exact behavior is different. Therefore, the concepts presented here can also be applied to VVC when the 4-tuples are adapted appropriately.

(100) To conclude, embodiments according to the invention provide an improved tradeoff between a compression performance and a visual quality and a low encoding latency, which leads to an improved coding efficiency. Some embodiments also provide an additional coding efficiency.

(101) Different inventive embodiments and aspects are described, for example, in the chapters "Introduction", "Sample Adaptive Offset", "Classification for PSAO", "Decoder", "Encoder" and "Some Comments", wherein features, functionalities and details from the chapter "Sample Adaptive Offset" may optionally be introduced into any of the other embodiments.

(102) However, the features, functionalities and details described in any other chapters can also,

optionally, be introduced into the embodiments according to the present invention.

(103) Also, the embodiments described in the above mentioned chapters can be used individually, and can also be supplemented by any of the features, functionalities and details in another chapter.

(104) Also, it should be noted that individual aspects described herein can be used individually or in combination. Thus, details can be added to each of said individual aspects without adding details to another one of said aspects.

(105) In particular, embodiments are also described in the claims. The embodiments described in the claims can optionally be supplemented by any of the features, functionalities and details as described herein, both individually and in combination.

(106) It should also be noted that the present disclosure describes, explicitly or implicitly, features usable in a video encoder (apparatus for providing an encoded representation of an input video signal) and in a video decoder (apparatus for providing a decoded representation of a video signal on the basis of an encoded representation of a video signal). Thus, any of the features described herein can be used in the context of a video encoder and in the context of a video decoder.

(107) Moreover, features and functionalities disclosed herein relating to a method can also be used in an apparatus (configured to perform such functionality). Furthermore, any features and functionalities disclosed herein with respect to an apparatus can also be used in a corresponding method. In other words, the methods disclosed herein can be supplemented by any of the features and functionalities described with respect to the apparatuses.

(108) Also, any of the features and functionalities described herein can be implemented in hardware or in software, or using a combination of hardware and software, as will be described in the section “implementation alternatives”.

Implementation Alternatives

(109) Although some aspects are described in the context of an apparatus, it is clear that these aspects also represent a description of the corresponding method, where a block or device corresponds to a method step or a feature of a method step. Analogously, aspects described in the context of a method step also represent a description of a corresponding block or item or feature of a corresponding apparatus. Some or all of the method steps may be executed by (or using) a hardware apparatus, like for example, a microprocessor, a programmable computer or an electronic circuit. In some embodiments, one or more of the most important method steps may be executed by such an apparatus.

(110) Depending on certain implementation requirements, embodiments of the invention can be implemented in hardware or in software. The implementation can be performed using a digital storage medium, for example a floppy disk, a DVD, a Blu-Ray, a CD, a ROM, a PROM, an EPROM, an EEPROM or a FLASH memory, having electronically readable control signals stored thereon, which cooperate (or are capable of cooperating) with a programmable computer system such that the respective method is performed. Therefore, the digital storage medium may be computer readable.

(111) Some embodiments according to the invention comprise a data carrier having electronically readable control signals, which are capable of cooperating with a programmable computer system, such that one of the methods described herein is performed.

(112) Generally, embodiments of the present invention can be implemented as a computer program product with a program code, the program code being operative for performing one of the methods when the computer program product runs on a computer. The program code may for example be stored on a machine readable carrier.

(113) Other embodiments comprise the computer program for performing one of the methods described herein, stored on a machine readable carrier.

(114) In other words, an embodiment of the inventive method is, therefore, a computer program having a program code for performing one of the methods described herein, when the computer program runs on a computer.

(115) A further embodiment of the inventive methods is, therefore, a data carrier (or a digital storage medium, or a computer-readable medium) comprising, recorded thereon, the computer program for performing one of the methods described herein. The data carrier, the digital storage medium or the recorded medium are typically tangible and/or non-transitory.

(116) A further embodiment of the inventive method is, therefore, a data stream or a sequence of signals representing the computer program for performing one of the methods described herein. The data stream or the sequence of signals may for example be configured to be transferred via a data communication connection, for example via the Internet.

(117) A further embodiment comprises a processing means, for example a computer, or a programmable logic device, configured to or adapted to perform one of the methods described herein.

(118) A further embodiment comprises a computer having installed thereon the computer program for performing one of the methods described herein.

(119) A further embodiment according to the invention comprises an apparatus or a system configured to transfer (for example, electronically or optically) a computer program for performing one of the methods described herein to a receiver. The receiver may, for example, be a computer, a mobile device, a memory device or the like. The apparatus or system may, for example, comprise a file server for transferring the computer program to the receiver.

(120) In some embodiments, a programmable logic device (for example a field programmable gate array) may be used to perform some or all of the functionalities of the methods described herein. In some embodiments, a field programmable gate array may cooperate with a microprocessor in order to perform one of the methods described herein. Generally, the methods are performed by any hardware apparatus.

(121) The apparatus described herein may be implemented using a hardware apparatus, or using a computer, or using a combination of a hardware apparatus and a computer.

(122) The apparatus described herein, or any components of the apparatus described herein, may be implemented at least partially in hardware and/or in software.

(123) The methods described herein may be performed using a hardware apparatus, or using a computer, or using a combination of a hardware apparatus and a computer.

(124) The methods described herein, or any components of the apparatus described herein, may be performed at least partially by hardware and/or by software.

(125) The herein described embodiments are merely illustrative for the principles of the present invention. It is understood that modifications and variations of the arrangements and the details described herein will be apparent to others skilled in the art. It is the intent, therefore, to be limited only by the scope of the impending patent claims and not by the specific details presented by way of description and explanation of the embodiments herein.

(126) Subsequently, embodiments are presented which represent broadenings of the embodiments set forth above, or differently speaking, embodiments, for which the above described embodiments represent specific examples with specific details, and for which these specific details may, individually or in combination, be used to further specify the subsequently presented embodiments.

(127) Text in brackets represents optional features, examples and descriptions. Optional features, examples described with respect to a feature of a specific embodiment may also apply to equivalent or similar features of the other embodiments. 1. Arithmetic encoder for arithmetically encoding a sequence of information values into an arithmetic coded bitstream, configured to: symbolize the information values into symbol strings so as to obtain a sequence of symbols; arithmetically encode the sequence of symbols by, for each symbol, selecting a subinterval out of a plurality of subintervals into which a current interval, which defines a current version of a coding state of the arithmetic encoder, is subdivided according to a probability estimate for the respective symbol, according to a symbol value of the respective symbol so as to obtain an updated version of the coding state of the arithmetic encoder, defined by the selected subinterval, for encoding a next

symbol of the sequence of symbols, and renormalizing encoder-internal parameters which define the coding state under continuing the bitstream, provide the bitstream with entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward.

2. The arithmetic encoder in accordance with embodiment 1, wherein the entry point information comprises information on an coding state of an arithmetic decoder, which occurs in the arithmetic decoder when the arithmetic decoder decodes the bitstream until the predetermined entry point.

3. The arithmetic encoder in accordance with embodiment 2, wherein the arithmetic encoder is configured to perform the arithmetic decoding the bitstream so as to determine the information on the coding state of the arithmetic decoder.

4. The arithmetic encoder in accordance with any of embodiments 2-3, wherein the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the information on the coding state of the arithmetic decoder comprises a value of the pointer.

5. The arithmetic encoder in accordance with embodiment 4, wherein the arithmetic encoder is configured to perform the arithmetic decoding the bitstream and set the value of the pointer comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the pointer as manifesting itself in the arithmetic decoding the bitstream up to the predetermined entry point.

6. The arithmetic encoder in accordance with any of embodiments 2-3, wherein the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the information on the coding state of the arithmetic decoder comprises a value of the interval width parameter.

7. The arithmetic encoder in accordance with embodiment 6, wherein the arithmetic encoder is configured to perform the arithmetic decoding the bitstream and set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the interval width parameter as manifesting itself in the arithmetic decoding the bitstream up to the predetermined entry point, or set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the interval width parameter as manifesting itself in the arithmetic encoding the sequence of symbols up to the predetermined entry point.

8. The arithmetic encoder in accordance with any of embodiments 2-3, wherein the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the information on the coding state of the arithmetic decoder comprises a value of the pointer [e.g. which value is assumed by the pointer at the predetermined entry point] while being free of a value of the interval width parameter, wherein the arithmetic encoder is configured to set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a predetermined value and use the predetermined value for the interval width parameter when resuming the arithmetic encoding the sequence of symbols from the predetermined entry point onwards, or preliminarily interrupting the arithmetic encoding the sequence of symbols by arithmetically encoding a symbol of a predetermined symbol value immediately ahead the predetermined entry point before resuming the arithmetic encoding the sequence of symbols from the predetermined entry point onwards, and set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the interval width parameter as manifesting itself in the arithmetic encoding the sequence of symbols up to the predetermined entry point, including the symbol of the predetermined symbol value.

9. The arithmetic encoder in accordance with any of previous embodiments, wherein the entry point information comprises a bitstream pointer to a predetermined bit in the bitstream, which is up to be read next after the resuming the arithmetic decoding the bitstream from the predetermined entry point onwards.

10. The arithmetic encoder in accordance with embodiment 9, wherein the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a begin of the bitstream.

11. The arithmetic encoder in accordance with embodiment 9,

wherein the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a trailing end of a run of leading bits of the bitstream based on which an coding state of the arithmetic decoder for performing the arithmetic decoding of the bitstream is to be initialized. 12. The arithmetic encoder in accordance with embodiment 9, wherein the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward, and the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a previous entry point or relative to a predefined bitstream position which is associated with the predetermined entry point. 13. The arithmetic encoder in accordance with embodiment 12, configured to locate the predefined bitstream position as a point between consecutive ones of the sequence of values or by counting bits of the bitstream. 14. The arithmetic encoder in accordance with embodiment 12, wherein the bitstream pointer to the predetermined bit in the bitstream is stored in units of bits, and/or an integer multiple n of bits, with $n > 1$, and $n = 8$, for example. 15. The arithmetic encoder in accordance with embodiment 12, wherein a bit position in the bitstream of the previous entry point is signaled in the bitstream. 16. The arithmetic encoder in accordance with any of embodiments 12 to 15, wherein the bitstream pointer is signaled in the bitstream differentially with respect to a further bit stream pointer which is comprised by the entry point information for pointing to a further predetermined bit in the bitstream, which is up to be read next after a resuming the arithmetic decoding the bitstream from a preceding entry point onwards. 17. The arithmetic encoder in accordance with any of embodiments 9-15, wherein the bitstream pointer is signaled in the bitstream using a variable length code. 18. The arithmetic encoder in accordance with any of embodiments 9-17, wherein the bitstream pointer is signaled in the bitstream using an Exponential-Golomb code, preferably an unsigned Exponential-Golomb code. 19. The arithmetic encoder in accordance with any of embodiments 9-18, wherein the predetermined entry point is a third or any of entry points following after the third entry point relative to the begin of the bitstream, and the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of a difference between the offset relative to the previous entry point and an offset of the previous entry point relative to a further entry point preceding the previous entry point. 20. The arithmetic encoder in accordance with embodiment 19, wherein the bitstream pointer is signaled in the bitstream using a signed Exponential-Golomb code. 21. The arithmetic encoder in accordance with any of embodiments 19 to 20, wherein the bit position in the bitstream of the previous entry point is signaled in the bitstream using an unsigned Exponential-Golomb code. 22. The arithmetic encoder in accordance with embodiment 21, wherein an exponential Golomb code parameter for the Exponential-Golomb code is a value of 11. 23. The arithmetic encoder in accordance with any of embodiments 19 to 21, wherein the bitstream pointer is signaled in the bitstream using a signed Exponential-Golomb code. 24. The arithmetic encoder in accordance with embodiment 23, wherein an exponential Golomb code parameter for the Exponential-Golomb code is a value of 7. 25. The arithmetic encoder in accordance with any of previous embodiments, wherein the arithmetic encoder is configured to use context-adaptive arithmetic coding for arithmetically encoding the sequence of symbols, including selecting, for a context-adaptively encoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols, wherein the entry point information is indicative of, for each of a set of one or more predetermined context models, a predetermined probability estimate for the respective predetermined context model, and the arithmetic encoder is configured to use the predetermined probability estimate in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model. 26. The arithmetic encoder in accordance with any of previous embodiments, wherein the arithmetic encoder is configured to use context-adaptive arithmetic coding for arithmetically encoding the sequence of symbols, including selecting, for a context-adaptively encoded symbol of

the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols, wherein the arithmetic encoder is configured to, for each of a set of one or more predetermined context models, set the probability estimate for the respective predetermined context model at the predetermined entry point to a default state and the arithmetic encoder is configured to use the default state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model, or wherein the arithmetic encoder is configured to, for each of a set of one or more predetermined context models, set the probability estimate for the respective predetermined context model at the predetermined entry point to a saved state manifesting itself at a predetermined condition during arithmetically encoding the sequence of symbols ahead the predetermined entry point and the arithmetic encoder is configured to use the saved state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model. 27. The arithmetic encoder in accordance with any of previous embodiments, wherein the arithmetic encoder is configured to use dependent quantization for deriving the sequence of information values from a sequence of unquantized values using state machine, wherein the entry point information comprises a quantization state manifesting itself in the state machine up to the predetermined entry point. 28. The arithmetic encoder in accordance with any of previous embodiments, wherein the symbols are bins and the symbolization is a binarization. 29. The arithmetic encoder in accordance with any of previous embodiments, wherein the information values is a sequence of syntax elements which represent a video. 30. The arithmetic encoder in accordance with any of embodiments 1-29, wherein the information values are neural network parameters. 31. Arithmetic decoder for arithmetically decoding a sequence of information values from a bitstream, configured to: derive from the bitstream entry point information; use the entry point information so as to resume arithmetically decoding the bitstream from a predetermined entry point onward by arithmetically decoding a sequence of symbols from the bitstream by, for each symbol of the bitstream: determining, based on a current version of a coding state of the arithmetic decoder, a subinterval out of a plurality of subintervals into which a current interval is subdivided according to a probability estimate for the respective symbol, and deduce, based on the selected subinterval, a symbol value of the respective symbol, and renormalizing and updating decoder-internal parameters which define the coding state by use of the bitstream and the selected subinterval so as to obtain an updated version of the coding state of the arithmetic decoder for decoding a next symbol of the sequence of symbols, and derive the information values from the sequence of symbols by desymbolization. 32. The arithmetic decoder in accordance with embodiment 31, configured to use the entry point information so as to determine a starting version of the coding state of the arithmetic decoder and use the starting state for commencing arithmetically decoding the bitstream from the predetermined entry point onwards. 33. The arithmetic decoder in accordance with any of embodiments 31 or 32, wherein the coding state of the arithmetic decoder is defined by the decoder-internal parameters which include an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the arithmetic decoder is configured to derive from the entry point information a starting value of the pointer using which the arithmetically decoding the bitstream from the predetermined entry point onwards is commenced. 34. The arithmetic decoder in accordance with any of embodiments 31-33, wherein the coding state of the arithmetic decoder is defined by the decoder-internal parameters which include an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the arithmetic decoder is configured to derive from the entry point information a starting value of the interval width parameter using which the arithmetically decoding the bitstream from the predetermined entry point onwards is commenced. 35. The arithmetic decoder in accordance with any of embodiments 31-33, wherein the coding state of the arithmetic decoder

is defined by the decoder-internal parameters which include an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the arithmetic decoder is configured to derive from the entry point information a starting value of the pointer using which the arithmetically decoding the bitstream from the predetermined entry point onwards is commenced and to set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a predetermined value and use the predetermined value for the interval width parameter for resuming the arithmetic decoding the sequence of symbols from the predetermined entry point onwards. 36. The arithmetic decoder in accordance with embodiment 35, wherein the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward and the arithmetic decoder is configured to preliminarily interrupting the arithmetic decoding the sequence of symbols at a subsequent predetermined entry point by arithmetically decoding a symbol of a predetermined symbol value immediately ahead the subsequent predetermined entry point before resuming the arithmetic decoding the sequence of symbols from the subsequent predetermined entry point onwards. 37. The arithmetic decoder in accordance with any of embodiments 31-36, configured to derive from the entry point information a bitstream pointer to a predetermined bit in the bitstream and use the predetermined bit as a bit to be read next after resuming the arithmetic decoding the bitstream from the predetermined entry point onwards. 38. The arithmetic decoder in accordance with embodiment 37, wherein the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a begin of the bitstream. 39. The arithmetic decoder in accordance with embodiment 37, wherein the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a trailing end of a run of leading bits of the bitstream based on which the arithmetic decoder is configured to initialize the coding state of the arithmetic decoder in case of performing the arithmetic decoding of the bitstream from the begin of the bitstream onwards. 40. The arithmetic decoder in accordance with embodiment 37, wherein the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward, and the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a previous entry point or relative to a predefined bitstream position which is associated with the predetermined entry point. 41. The arithmetic decoder in accordance with embodiment 40, configured to locate the predefined bitstream position as a point between consecutive ones of the sequence of values or by counting bits of the bitstream. 42. The arithmetic decoder in accordance with embodiment 40, wherein the bitstream pointer to the predetermined bit in the bitstream is stored in units of bits, and/or an integer multiple n of bits, with $n > 1$, and $n = 8$, for example. 43. The arithmetic decoder in accordance with embodiment 42, wherein a bit position in the bitstream of the previous entry point is signaled in the bitstream. 44. The arithmetic decoder in accordance with any of embodiments 40 to 43, wherein the bitstream pointer is signaled in the bitstream differentially with respect to a further bit stream pointer which is comprised by the entry point information for pointing to a further predetermined bit in the bitstream, which is up to be read next after a resuming the arithmetic decoding the bitstream from a preceding entry point onwards. 45. The arithmetic decoder in accordance with any of embodiments 37-43, wherein the bitstream pointer is signaled in the bitstream using a variable length code. 46. The arithmetic decoder in accordance with any of embodiments 37-44, wherein the bitstream pointer is signaled in the bitstream using an Exponential-Golomb code, preferably an unsigned Exponential-Golomb code. 47. The arithmetic decoder in accordance with any of embodiments 37-46, wherein the predetermined entry point is a third or any of entry points following after the third entry point relative to the begin of the bitstream, and the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of a difference between the offset relative to the previous entry point and an offset of the previous entry point relative to a further entry point preceding the previous entry point. 48. The arithmetic decoder in accordance with embodiment 47, wherein the bitstream pointer is

signaled in the bitstream using a signed Exponential-Golomb code. 49. The arithmetic decoder in accordance with any of embodiments 47 to 48, wherein the bit position in the bitstream of the previous entry point is signaled in the bitstream using an unsigned Exponential-Golomb code. 50. The arithmetic decoder in accordance with embodiment 49, wherein an exponential Golomb code parameter for the Exponential-Golomb code is a value of 11. 51. The arithmetic decoder in accordance with any of embodiments 47 to 50, wherein the bitstream pointer is signaled in the bitstream using a signed Exponential-Golomb code. 52. The arithmetic decoder in accordance with embodiment 51, wherein an exponential Golomb code parameter for the Exponential-Golomb code is a value of 7. 53. The arithmetic decoder in accordance with any of embodiments 31-52, wherein the arithmetic decoder is configured to use context-adaptive arithmetic decoding for arithmetically decoding the sequence of symbols, including selecting, for a context-adaptively decoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols, wherein the entry point information is indicative of, for each of a set of one or more predetermined context models, a predetermined probability estimate for the respective predetermined context model, and the arithmetic decoder is configured to use the predetermined probability estimate in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model. 54. The arithmetic decoder in accordance with any of embodiments 31-53, wherein the arithmetic decoder is configured to use context-adaptive arithmetic decoding for arithmetically decoding the sequence of symbols, including selecting, for a context-adaptively decoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols, wherein the arithmetic decoder is configured to, for each of a set of one or more predetermined context models, set the probability estimate for the respective predetermined context model at the predetermined entry point to a default state and the arithmetic decoder is configured to use the default state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model. 55. The arithmetic decoder in accordance with embodiment 54, wherein the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward and the arithmetic decoder is configured to for each of the set of one or more predetermined context models, set the probability estimate for the respective predetermined context model at a subsequent predetermined entry point to a saved state manifesting itself at a predetermined condition during arithmetically decoding the sequence of symbols ahead the subsequent predetermined entry point and the arithmetic decoder is configured to use the saved state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model from the subsequent predetermined entry point onwards. 56. The arithmetic decoder in accordance with any of embodiments 31-55, wherein the arithmetic decoder is configured to use dependent dequantization for deriving from the sequence of information values a sequence of quantized values using a state machine, derive from the entry point information a quantization state starting with which the dependent quantization is resumed from the predetermined entry point onwards. 57. The arithmetic decoder in accordance with any of embodiments 31-56, wherein the symbols are bins and the desymbolization is a debinarization. 58. The arithmetic decoder in accordance with any of embodiments 31-57, wherein the information values is a sequence of syntax elements which represent a video. 59. The arithmetic decoder in accordance with any of embodiments 31-57, wherein the information values are neural network parameters. 60. Arithmetic decoder for arithmetically decoding neural network parameters from a bitstream, configured to: arithmetically decode a sequence of symbols from the bitstream by use of context-adaptive arithmetic decoding,

including selecting, for a context-adaptively decoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith and arithmetically decoding the context-adaptively decoded symbol using the selected context model, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols, derive the neural network parameters from the sequence of symbols by desymbolization, at the beginning of the bitstreams and/or at one or more entry points within the bitstream, initialize, for each of a set of one or more context models, the probability estimate associated with the respective context model based on context model information in the bitstream. 61. Arithmetic decoder of embodiment 60, configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols by for each context model, generating the probability estimate associated with the respective context model on the basis of previously context-adaptively decoded symbols of the sequence of symbols for which the respective context model has been selected. 62. Arithmetic decoder of embodiment 60 or 61, configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols by deriving a first hypothesis for the probability estimates at a manner which adapts to the actual symbol statistics at a first adaptation agility which is controllable by a first agility parameter, at the beginning of the bitstream and/or at one or more entry points within the bitstream, set, for each of the set of one or more context models, the first hypothesis and the first agility parameter associated with the respective context model based on the context model information in the bitstream. 63. Arithmetic decoder of embodiment 62, configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols by deriving a second hypothesis for the probability estimates at a manner which adapts to the actual symbol statistics at a second adaptation agility which is controllable by a second agility parameter, wherein the probability estimates are determined by an average of the first and second hypotheses, at the beginning of the bitstream and/or at one or more entry points within the bitstream, set, for each of the set of one or more context models, the second hypothesis and the second agility parameter associated with the respective context model based on the context model information in the bitstream. 64. Arithmetic decoder of embodiment 63, wherein the context model information in the bitstream comprises a table entry index into a table of quadruples of values for defining the first and second hypotheses and the first and second agility parameters, and the arithmetic decoder is configured to use the table entry index so as to select one quadruple of the table and to use the one quadruple so as to set the first and second hypotheses and the first and second agility parameters. 65. Arithmetic decoder of embodiment 64, wherein a number of the quadruples of values for defining the first and second hypotheses and the first and second agility parameters lies between 8 and 10, both inclusively. 66. Arithmetic decoder of any of embodiments 64 and 65, wherein the quadruples of values for defining the first and second hypotheses and the first and second agility parameters correspond to one of 3, 4 or 5 mutually distinguishable settings for the first and second agility parameters. 67. Arithmetic decoder of any of embodiments 64 to 66, wherein the symbols are bins and the desymbolization is a debinarization and the quadruples of values for defining the first and second hypotheses and the first and second agility parameters comprise first 3 quadruples according to all of which the first agility parameter is set to a first value, the second agility parameter is set to a second value, which corresponds to lower adaptation agility than the first value, and according to a first one of which the first and second hypotheses correspond to equiprobability, according to a second one of which the first and second hypotheses correspond to a first bin value being more probable than a second bin value, and according to a third one of which the first and second hypotheses correspond to the second bin value being more probable than the first bin value, second 3 quadruples according to all of which the first agility parameter is set to a third value, which corresponds to lower adaptation agility than the first value, and larger adaptation

agility than the second value, and the second agility parameter is set to a fourth value, which corresponds to lower adaptation agility than the second value, and according to a first and second one of which the first and second hypotheses correspond to the first bin value being more probable than the second bin value, and according to a third one of which the first and second hypotheses correspond to the second bin value being more probable than the first bin value, 2 quadruples according to all of which the first agility parameter is set to a fourth value, which corresponds to lower adaptation agility than the third value, and larger adaptation agility than the second value, and the second agility parameter is set to a sixth value, which corresponds to lower adaptation agility than the second value, and larger adaptation agility than the fourth value, and according to a first one of which the first and second hypotheses correspond to equi-probability, and according to a second one of which the first and second hypotheses correspond to the first bin value being more probable than the second bin value, 1 quadruple according to which the first agility parameter is set to a seventh value, which corresponds to larger adaptation agility than the first value, and according to which the first and second hypotheses correspond to equi-probability. 68. Arithmetic decoder of any of embodiments 63 to 67, wherein the symbols are bins and the desymbolization is a debinarization and the arithmetic decoder is configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols by, for each of the first and second hypotheses, representing the respective hypothesis by means of a signed integer, respectively, which, when being zero, is indicative of equi-probability, which, when being larger than zero, is indicative of a first bin value being more probable than a second bin value, and which, when being lower than zero, is indicative of the second bin value being more probable than the first bin value, for each context model, increase the signed integer if a currently decoded bin has the first bin value, and decrease the signed integer if a currently decoded bin has the second bin value, at an amount which is controlled by the first agility parameter with respect to the first hypothesis and controlled by the second agility parameter with respect to the second hypothesis so that the amount is the larger the smaller the first and second agility parameter is, respectively, wherein the probability estimates are determined by an average of the first and second signed integers. 69. Arithmetic decoder of embodiment 68, configured to determine the amount for the increase and decrease by use of a transition table. 70. Arithmetic decoder of embodiment 69, configured to use the same transition table for the first and second hypotheses. 71. Arithmetic decoder of embodiment 69 or 70, configured to determine the amount for the increase and decrease by use of the transition table by looking up the transition table at an entry indexed by a table index determined by the signed integer to obtain a transition step and dividing the transition step size by 2 to a power which depends on the first and second adaptation parameter, respectively, wherein the transition step determines the amount. 72. Arithmetic decoder of embodiment 69 or 70, wherein the signed integer is represented by a 2's-complement representation with n bits, with n being larger for the second hypothesis than for the first hypothesis, wherein the arithmetic decoder is configured to determine the amount for the increase and decrease by use of the transition table by looking up the transition table at an entry indexed by a sum of the signed integer divided by $2^{\sup.n-m}$ on the one hand and $2^{\sup.m-1}$ on the other hand in case to obtain a transition step dividing the transition step size by 2 to a power which affine linearly depends on the first and second adaptation parameter, respectively, wherein the transition step determines the amount. 73. Arithmetic decoder of embodiment 72, wherein the transition steps stored in the entries of the transition table increase or decrease monotonically. 74. A method (100) of arithmetically encoding a sequence of information values into an arithmetic coded bitstream, comprising: symbolizing (101) the information values into symbol strings so as to obtain a sequence of symbols; arithmetically encoding (102) the sequence of symbols by subdividing (103) a current interval, which defines a current version of an coding state of an arithmetic encoder, for each symbol, according to a probability estimate for the respective symbol, selecting (104) a subinterval out of a plurality of the subintervals according to a symbol value of the respective

symbol so as to obtain an updated version of the coding state of the arithmetic encoder, defined by the selected subinterval, for encoding a next symbol of the sequence of symbols, and renormalizing (105) encoder-internal parameters which define the coding state under continuing the bitstream, providing (106) the bitstream with entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward. 75. The method in accordance with embodiment 74, wherein the entry point information comprises information on an coding state of an arithmetic decoder, which occurs in the arithmetic decoder when the arithmetic decoder decodes the bitstream until the predetermined entry point. 76. The method in accordance with embodiment 75, further comprising performing the arithmetic decoding the bitstream so as to determine the information on the coding state of the arithmetic decoder. 77. The method in accordance with any of embodiments 75-76, wherein the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the information on the coding state of the arithmetic decoder comprises a value of the pointer. 78. The method in accordance with embodiment 77, further comprising performing the arithmetic decoding the bitstream and setting the value of the pointer comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the pointer as manifesting itself in the arithmetic decoding the bitstream up to the predetermined entry point. 79. The method in accordance with any of embodiments 75-76, wherein the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the information on the coding state of the arithmetic decoder comprises a value of the interval width parameter. 80. The method in accordance with embodiment 79, further comprising performing the arithmetic decoding the bitstream and setting the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the interval width parameter as manifesting itself in the arithmetic decoding the bitstream up to the predetermined entry point, or setting the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the interval width parameter as manifesting itself in the arithmetic encoding the sequence of symbols up to the predetermined entry point. 81. The method in accordance with any of embodiments 75-76, wherein the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the information on the coding state of the arithmetic decoder comprises a value of the pointer while being free of a value of the interval width parameter, wherein the method further comprises setting the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a predetermined value and using the predetermined value for the interval width parameter when resuming the arithmetic encoding the sequence of symbols from the predetermined entry point onwards, or preliminarily interrupting the arithmetic encoding the sequence of symbols by arithmetically encoding a symbol of a predetermined symbol value immediately ahead the predetermined entry point before resuming the arithmetic encoding the sequence of symbols from the predetermined entry point onwards, and setting the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the interval width parameter as manifesting itself in the arithmetic encoding the sequence of symbols up to the predetermined entry point, including the symbol of the predetermined symbol value. 82. The method in accordance with any of embodiments 74-81, wherein the entry point information comprises a bitstream pointer to a predetermined bit in the bitstream, which is up to be read next after the resuming the arithmetic decoding the bitstream from the predetermined entry point onwards. 83. The method in accordance with embodiment 82, wherein the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a begin of the bitstream. 84. The method in accordance with embodiment 82, wherein the bitstream

pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a trailing end of a run of leading bits of the bitstream based on which an coding state of an arithmetic decoder for performing the arithmetic decoding of the bitstream is to be initialized.

85. The method in accordance with embodiment 82, wherein the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward, and the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a previous entry point or relative to a predefined bitstream position which is associated with the predetermined entry point.

86. The method in accordance with embodiment 85, comprising locating the predefined bitstream position as a point between consecutive ones of the sequence of values or by counting bits of the bitstream.

87. The method in accordance with embodiment 85, wherein the bitstream pointer to the predetermined bit in the bitstream is stored in units of bits, and/or an integer multiple n of bits, with $n > 1$, and $n = 8$, for example.

88. The method in accordance with embodiment 85, wherein a bit position in the bitstream of the previous entry point is signaled in the bitstream.

89. The method in accordance with any of embodiments 85 to 88, wherein the bitstream pointer is signaled in the bitstream differentially with respect to a further bit stream pointer which is comprised by the entry point information for pointing to a further predetermined bit in the bitstream, which is up to be read next after a resuming the arithmetic decoding the bitstream from a preceding entry point onwards.

90. The method in accordance with any of embodiments 82-89, wherein the bitstream pointer is signaled in the bitstream using a variable length code.

91. The method in accordance with any of embodiments 82-90, wherein the bitstream pointer is signaled in the bitstream using an Exponential-Golomb code, preferably an unsigned Exponential-Golomb code.

92. The method in accordance with any of embodiments 82-91, wherein the predetermined entry point is a third or any of entry points following after the third entry point relative to the begin of the bitstream, and the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of a difference between the offset relative to the previous entry point and an offset of the previous entry point relative to a further entry point preceding the previous entry point.

93. The method in accordance with embodiment 92, wherein the bitstream pointer is signaled in the bitstream using a signed Exponential-Golomb code.

94. The method in accordance with any of embodiments 92 to 93, wherein the bit position in the bitstream of the previous entry point is signaled in the bitstream using an unsigned Exponential-Golomb code.

95. The method in accordance with embodiment 94, wherein an exponential Golomb code parameter for the Exponential-Golomb code is a value of 11.

96. The method in accordance with any of embodiments 92 to 95, wherein the bitstream pointer is signaled in the bitstream using a signed Exponential-Golomb code.

97. The method in accordance with embodiment 96, wherein an exponential Golomb code parameter for the Exponential-Golomb code is a value of 7.

98. A method **(200)** of arithmetically decoding a sequence of information values from a bitstream, comprising: deriving **(201)** from the bitstream entry point information; using **(202)** the entry point information so as to resume arithmetically decoding the bitstream from a predetermined entry point onward by arithmetically decoding a sequence of symbols from the bitstream by, for each symbol of the bitstream: determining **(203)**, a current version of an coding state of an arithmetic decoder, a subinterval out of a plurality of subintervals into which a current interval is subdivided according to a probability estimate for the respective symbol, and deducing **(204)**, based on the selected subinterval, a symbol value of the respective symbol, and renormalizing and updating **(205)** decoder-internal parameters which define the coding state by use of the bitstream and the selected subinterval so as to obtain an updated version of the coding state of the arithmetic decoder for decoding a next symbol of the sequence of symbols, and deriving **(206)** the information values from the sequence of symbols by desymbolization.

99. The method in accordance with embodiment 98, further comprising using the entry point information so as to determine a starting version of the coding state of the arithmetic decoder and using the starting state for commencing arithmetically decoding the bitstream from the predetermined entry point onwards.

100. The

method in accordance with any of embodiments 98 or 99, wherein the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the method further comprises deriving from the entry point information a starting value of the pointer using which the arithmetically decoding the bitstream from the predetermined entry point onwards is commenced.

101. The method in accordance with any of embodiments 98-100, wherein the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the method further comprises deriving from the entry point information a starting value of the interval width parameter using which the arithmetically decoding the bitstream from the predetermined entry point onwards is commenced.

102. The method in accordance with any of embodiments 98-100, wherein the coding state of the arithmetic decoder is defined by decoder-internal parameters including an interval width parameter indicating a width of the interval and a pointer pointing into the interval and the method further comprises: deriving from the entry point information a starting value of the pointer using which the arithmetically decoding the bitstream from the predetermined entry point onwards is commenced and setting the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a predetermined value and use the predetermined value for the interval width parameter for resuming the arithmetic decoding the sequence of symbols from the predetermined entry point onwards.

103. The method in accordance with embodiment 102, wherein the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward and the method further comprises preliminarily interrupting the arithmetic decoding the sequence of symbols at a subsequent predetermined entry point by arithmetically decoding a symbol of a predetermined symbol value immediately ahead the subsequent predetermined entry point before resuming the arithmetic encoding the sequence of symbols from the subsequent predetermined entry point onwards.

104. The method in accordance with any of embodiments 98-103, further comprising deriving from the entry point information a bitstream pointer to a predetermined bit in the bitstream and using the predetermined bit as a bit to be read next after resuming the arithmetic decoding the bitstream from the predetermined entry point onwards.

105. The method in accordance with embodiment 104, wherein the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a begin of the bitstream.

106. The method in accordance with embodiment 104, wherein the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a trailing end of a run of leading bits of the bitstream based on which the method further initializes the coding state of the arithmetic decoder in case of performing the arithmetic decoding of the bitstream from the begin of the bitstream onwards.

107. The method in accordance with embodiment 104, wherein the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward, and the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of an offset relative to a previous entry point or relative to a predefined bitstream position which is associated with the predetermined entry point.

108. The method in accordance with embodiment 107, comprising locating the predefined bitstream position as a point between consecutive ones of the sequence of values or by counting bits of the bitstream.

109. The method in accordance with embodiment 107, wherein the bitstream pointer to the predetermined bit in the bitstream is stored in units of bits, and/or an integer multiple n of bits, with $n > 1$, and $n = 8$, for example.

110. The method in accordance with embodiment 107, wherein a bit position in the bitstream of the previous entry point is signaled in the bitstream.

111. The method in accordance with any of embodiments 104 to 110, wherein the bitstream pointer is signaled in the bitstream differentially with respect to a further bit stream pointer which is comprised by the entry point information for pointing to a further predetermined bit in the bitstream, which is up to be read next after a resuming the arithmetic decoding the bitstream from a preceding entry point onwards.

112.

The method in accordance with any of embodiments 104-111, wherein the bitstream pointer is signaled in the bitstream using a variable length code. 113. The method in accordance with any of embodiments 104-112, wherein the bitstream pointer is signaled in the bitstream using an Exponential-Golomb code, preferably an unsigned Exponential-Golomb code. 114. The method in accordance with any of embodiments 104-113, wherein the predetermined entry point is a third or any of entry points following after the third entry point relative to the begin of the bitstream, and the bitstream pointer to the predetermined bit in the bitstream is signaled in the bitstream in form of a difference between the offset relative to the previous entry point and an offset of the previous entry point relative to a further entry point preceding the previous entry point. 115. The method in accordance with embodiment 114, wherein the bitstream pointer is signaled in the bitstream using a signed Exponential-Golomb code. 116. The method in accordance with any of embodiments 114 to 115, wherein the bit position in the bitstream of the previous entry point is signaled in the bitstream using an unsigned Exponential-Golomb code. 117. The method in accordance with embodiment 116, wherein an exponential Golomb code parameter for the Exponential-Golomb code is a value of 11. 118. The method in accordance with any of embodiments 114 to 117, wherein the bitstream pointer is signaled in the bitstream using a signed Exponential-Golomb code. 119. The method in accordance with embodiment 118, wherein an exponential Golomb code parameter for the Exponential-Golomb code is a value of 7. 120. The method in accordance with any of embodiments 98-119, further comprising using context-adaptive arithmetic decoding for arithmetically decoding the sequence of symbols, including selecting, for a context-adaptively decoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols, wherein the entry point information is indicative of, for each of a set of one or more predetermined context models, a predetermined probability estimate for the respective predetermined context model, and the method further comprises using the predetermined probability estimate in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model. 121. The method in accordance with any of embodiments 98-120, further comprising using context-adaptive arithmetic decoding for arithmetically decoding the sequence of symbols, including selecting, for a context-adaptively decoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols, wherein the method further comprises, for each of a set of one or more predetermined context models, setting the probability estimate for the respective predetermined context model at the predetermined entry point to a default state and the method further comprises using the default state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model. 122. The method in accordance with embodiment 121, wherein the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward and the method further comprises for each of the set of one or more predetermined context models, setting the probability estimate for the respective predetermined context model at a subsequent predetermined entry point to a saved state manifesting itself at a predetermined condition during arithmetically decoding the sequence of symbols ahead the subsequent predetermined entry point and the method further comprises using the saved state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model from the subsequent predetermined entry point onwards. 123. The method in accordance with any of embodiments 98-122, further comprising: using dependent dequantization for deriving from the sequence of information values a sequence of quantized values using a state machine, deriving from the entry point information a quantization state starting with which the dependent

quantization is resumed from the predetermined entry point onwards. 124. The method in accordance with any of embodiments 98-123, wherein the symbols are bins and the desymbolization is a debinarization. 125. The method in accordance with any of embodiments 98-124, wherein the information values is a sequence of syntax elements which represent a video. 126. The method in accordance with any of embodiments 98-124, wherein the information values are neural network parameters. 127. Arithmetic encoder for arithmetically encoding neural network parameters into a bitstream, configured to: derive from the neural network parameters a sequence of symbols by symbolization, arithmetically encode the sequence of symbols into the bitstream by use of context-adaptive arithmetic encoding, including selecting, for a context-adaptively encoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith and arithmetically encoding the context-adaptively encoded symbol using the selected context model, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols, at the beginning of the bitstreams and/or at one or more entry points within the bitstream, initialize, for each of a set of one or more context models, the probability estimate associated with the respective context model according to context model information signaled in the bitstream. 128. Arithmetic encoder of embodiment 127, configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols by for each context model, generating the probability estimate associated with the respective context model on the basis of previously context-adaptively encoded symbols of the sequence of symbols for which the respective context model has been selected. 129. Arithmetic encoder of embodiment 127 or 128 configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols by deriving a first hypothesis for the probability estimates at a manner which adapts to the actual symbol statistics at a first adaptation agility which is controllable by a first agility parameter, at the beginning of the bitstreams and/or at one or more entry points within the bitstream, set, for each of the set of one or more context models, the first hypothesis and the first agility parameter associated with the respective context model according to the context model information signaled in the bitstream. 130. Arithmetic encoder of embodiment 127, configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols by deriving a second hypothesis for the probability estimates at a manner which adapts to the actual symbol statistics at a second adaptation agility which is controllable by a second agility parameter and forming an average of the first and second hypothesis, at the beginning of the bitstream and/or at one or more entry points within the bitstream, set, for each of the set of one or more context models, the second hypothesis and the second agility parameter associated with the respective context model according to the context model information signaled in the bitstream. 131. Arithmetic encoder of embodiment 130, wherein the context model information in the bitstream comprises a table entry index into a table of quadruples of values for defining the first and second hypotheses and the first and second agility parameters, and the arithmetic encoder is configured to use the table entry index so as to select one quadruple of the table and to use the one quadruple so as to set the first and second hypotheses and the first and second agility parameters. 132. Arithmetic encoder of embodiment 130, wherein a number of the quadruples of values for defining the first and second hypotheses and the first and second agility parameters lies between 8 and 10, both inclusively. 133. Arithmetic encoder of any of embodiments 131 and 132, wherein the quadruples of values for defining the first and second hypotheses and the first and second agility parameters correspond to one of 3, 4 or 5 mutually distinguishable settings for the first and second agility parameters. 134. Arithmetic encoder of any of embodiments 131 to 133, wherein the quadruples of values for defining the first and second hypotheses and the first and second agility parameters comprise first 3 quadruples according to all of which the first agility parameter is set to a first

value, the second agility parameter is set to a second value, which corresponds to lower adaptation agility than the first value, and according to a first one of which the first and second hypotheses correspond to equi-probability, according to a second one of which the first and second hypotheses correspond to a first bin value being more probable than a second bin value, and according to a third one of which the first and second hypotheses correspond to the second bin value being more probable than the first bin value, second 3 quadruples according to all of which the first agility parameter is set to a third value, which corresponds to lower adaptation agility than the first value, and larger adaptation agility than the second value, and the second agility parameter is set to a fourth value, which corresponds to lower adaptation agility than the second value, and according to a first and second one of which the first and second hypotheses correspond to the first bin value being more probable than the second bin value, and according to a third one of which the first and second hypotheses correspond to the second bin value being more probable than the first bin value, 2 quadruples according to all of which the first agility parameter is set to a fourth value, which corresponds to lower adaptation agility than the third value, and larger adaptation agility than the second value, and the second agility parameter is set to a sixth value, which corresponds to lower adaptation agility than the second value, and larger adaptation agility than the fourth value, and according to a first one of which the first and second hypotheses correspond to equi-probability, and according to a second one of which the first and second hypotheses correspond to the first bin value being more probable than the second bin value, 1 quadruple according to which the first agility parameter is set to a seventh value, which corresponds to larger adaptation agility than the first value, and according to which the first and second hypotheses correspond to equi-probability.

135. Arithmetic encoder of any of embodiments 130 to 134, wherein the symbols are bins and the symbolization is a binarization and the arithmetic encoder is configured to adapt the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols by, for each of the first and second hypotheses, representing the respective hypothesis by means of a signed integer, respectively, which, when being zero, is indicative of equi-probability, which, when being larger than zero, is indicative of a first bin value being more probable than a second bin value, and which, when being lower than zero, is indicative of the second bin value being more probable than the first bin value, for each context model, increase the signed integer if a currently encoded bin has the first bin value, and decrease the signed integer if a currently encoded bin has the second bin value, at an amount which is controlled by the first agility parameter with respect to the first hypothesis and controlled by the second agility parameter with respect to the second hypothesis so that the amount is the larger the smaller the first and second agility parameter is, respectively, wherein the probability estimates are determined by an average of the first and second signed integers.

136. Arithmetic encoder of embodiment 135, configured to determine the amount for the increase and decrease by use of a transition table.

137. Arithmetic encoder of embodiment 136, configured to use the same transition table for the first and second hypotheses.

138. Arithmetic encoder of embodiment 136 or 137, configured to determine the amount for the increase and decrease by use of the transition table by looking up the transition table at an entry indexed by a table index determined by the signed integer to obtain a transition step and dividing the transition step size by 2 to a power which depends on the first and second adaptation parameter, respectively, wherein the transition step determines the amount.

139. Arithmetic encoder of embodiment 136 or 137, wherein the signed integer is represented by a 2's-complement representation with n bits, with n being larger for the second hypothesis than for the first hypothesis, wherein the arithmetic encoder is configured to determine the amount for the increase and decrease by use of the transition table by looking up the transition table at an entry indexed by a sum of the signed integer divided by $2^{\sup.n-m}$ on the one hand and $2^{\sup.m-1}$ on the other hand in case to obtain a transition step dividing the transition step size by 2 to a power which affine linearly depends on the first and second adaptation parameter, respectively, wherein the transition step determines the amount.

140. Arithmetic encoder of embodiment 139, wherein the

transition steps stored in the entries of the transition table increase or decrease monotonically. 141. Computer program having a program code for performing, when running on a computer, methods according to any of embodiments 74-126. 142. A bitstream generated using an arithmetic encoder of any of embodiments 1-30, 127-140.

(128) While this invention has been described in terms of several embodiments, there are alterations, permutations, and equivalents which fall within the scope of this invention. It should also be noted that there are many alternative ways of implementing the methods and compositions of the present invention. It is therefore intended that the following appended claims be interpreted as including all such alterations, permutations and equivalents as fall within the true spirit and scope of the present invention.

Claims

1. An arithmetic encoder for arithmetically encoding a sequence of information values into an arithmetic coded bitstream, configured to: symbolize the information values into symbol strings so as to acquire a sequence of symbols; arithmetically encode the sequence of symbols by, for each symbol, selecting a subinterval out of a plurality of subintervals into which a current interval, which defines a current version of a coding state of the arithmetic encoder, is subdivided according to a probability estimate for the respective symbol, according to a symbol value of the respective symbol so as to acquire an updated version of the coding state of the arithmetic encoder, defined by the selected subinterval, for encoding a next symbol of the sequence of symbols, and renormalizing encoder-internal parameters which define the coding state under continuing the bitstream, provide the bitstream with entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward.
2. The arithmetic encoder in accordance with claim 1, wherein the entry point information comprises information on a coding state of an arithmetic decoder, which occurs in the arithmetic decoder when the arithmetic decoder decodes the bitstream until the predetermined entry point.
3. The arithmetic encoder in accordance with claim 2, wherein the arithmetic encoder is configured to perform the arithmetic decoding the bitstream so as to determine the information on the coding state of the arithmetic decoder.
4. The arithmetic encoder in accordance with claim 2, wherein the coding state of the arithmetic decoder is defined by decoder-internal parameters comprising an interval width parameter indicating a width of the current interval and a pointer pointing into the current interval and the information on the coding state of the arithmetic decoder comprises a value of the pointer.
5. The arithmetic encoder in accordance with claim 4, wherein the arithmetic encoder is configured to perform the arithmetic decoding the bitstream and set the value of the pointer comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the pointer as manifesting itself in the arithmetic decoding the bitstream up to the predetermined entry point.
6. The arithmetic encoder in accordance with claim 2, wherein the coding state of the arithmetic decoder is defined by decoder-internal parameters comprising an interval width parameter indicating a width of the current interval and a pointer pointing into the current interval and the information on the coding state of the arithmetic decoder comprises a value of the interval width parameter.
7. The arithmetic encoder in accordance with claim 6, wherein the arithmetic encoder is configured to perform the arithmetic decoding the bitstream and set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the interval width parameter as manifesting itself in the arithmetic decoding the bitstream up to the predetermined entry point, or set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the

interval width parameter as manifesting itself in the arithmetic encoding the sequence of symbols up to the predetermined entry point.

8. The arithmetic encoder in accordance with claim 2, wherein the coding state of the arithmetic decoder is defined by decoder-internal parameters comprising an interval width parameter indicating a width of the current interval and a pointer pointing into the current interval and the information on the coding state of the arithmetic decoder comprises a value of the pointer [e.g. which value is assumed by the pointer at the predetermined entry point] while being free of a value of the interval width parameter, wherein the arithmetic encoder is configured to set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a predetermined value and use the predetermined value for the interval width parameter when resuming the arithmetic encoding the sequence of symbols from the predetermined entry point onwards, or preliminarily interrupting the arithmetic encoding the sequence of symbols by arithmetically encoding a symbol of a predetermined symbol value immediately ahead the predetermined entry point before resuming the arithmetic encoding the sequence of symbols from the predetermined entry point onwards, and set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a current value of the interval width parameter as manifesting itself in the arithmetic encoding the sequence of symbols up to the predetermined entry point, comprising the symbol of the predetermined symbol value.

9. The arithmetic encoder in accordance with claim 2, wherein the entry point information comprises a bitstream pointer to a predetermined bit in the bitstream, which is up to be read next after the resuming the arithmetic decoding the bitstream from the predetermined entry point onwards.

10. The arithmetic encoder in accordance with claim 1, wherein the arithmetic encoder is configured to use context-adaptive arithmetic coding for arithmetically encoding the sequence of symbols, comprising selecting, for a context-adaptively encoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols, wherein the entry point information is indicative of, for each of a set of one or more predetermined context models, a predetermined probability estimate for the respective predetermined context model, and the arithmetic encoder is configured to use the predetermined probability estimate in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model.

11. The arithmetic encoder in accordance with claim 1, wherein the arithmetic encoder is configured to use context-adaptive arithmetic coding for arithmetically encoding the sequence of symbols, comprising selecting, for a context-adaptively encoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously encoded symbols of the sequence of symbols, wherein the arithmetic encoder is configured to, for each of a set of one or more predetermined context models, set the probability estimate for the respective predetermined context model at the predetermined entry point to a default state and the arithmetic encoder is configured to use the default state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model, or wherein the arithmetic encoder is configured to, for each of a set of one or more predetermined context models, set the probability estimate for the respective predetermined context model at the predetermined entry point to a saved state manifesting itself at a predetermined condition during arithmetically encoding the sequence of symbols ahead the predetermined entry point and the arithmetic encoder is configured to use the saved state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model.

12. The arithmetic encoder in accordance with claim 1, wherein the arithmetic encoder is configured to use dependent quantization for deriving the sequence of information values from a sequence of unquantized values using state machine, wherein the entry point information comprises a quantization state manifesting itself in the state machine up to the predetermined entry point.
13. The arithmetic encoder in accordance with claim 1, wherein the symbols are bins and the symbolization is a binarization.
14. The arithmetic encoder in accordance with claim 1, wherein the information values is a sequence of syntax elements which represent a video.
15. The arithmetic encoder in accordance with claim 1, wherein the information values are neural network parameters.
16. The arithmetic encoder in accordance with claim 1, wherein the arithmetic encoder is configured to provide the bitstream with the entry point information by signaling the entry point information in a header preceding the bitstream, following the bitstream or inserted in between the bitstream.
17. The arithmetic encoder in accordance with claim 1, wherein the entry point information comprises information on a coding state of an arithmetic decoder, which occurs in the arithmetic decoder when the arithmetic decoder decodes the bitstream until the predetermined entry point, the coding state of the arithmetic decoder being defined by the interval width of the current interval and a pointer pointing into the current interval, the information on the coding state of the arithmetic decoder comprises a value of the pointer as manifesting itself in the arithmetic decoding the bitstream up to the predetermined entry point, the arithmetic encoder is configured to provide the bitstream with the entry point information by signaling the entry point information in a header preceding the bitstream, following the bitstream or inserted in between the bitstream.
18. A bitstream generated using an arithmetic encoder of claim 1.
19. An arithmetic decoder for arithmetically decoding a sequence of information values from a bitstream, configured to: derive from the bitstream entry point information; use the entry point information so as to resume arithmetically decoding the bitstream from a predetermined entry point onward by arithmetically decoding a sequence of symbols from the bitstream by, for each symbol of the bitstream: determining, based on a current version of a coding state of the arithmetic decoder, a selected subinterval out of a plurality of subintervals into which a current interval is subdivided according to a probability estimate for the respective symbol, and deduce, based on the selected subinterval, a symbol value of the respective symbol, and renormalizing and updating decoder-internal parameters which define the coding state by use of the bitstream and the selected subinterval so as to acquire an updated version of the coding state of the arithmetic decoder for decoding a next symbol of the sequence of symbols, and derive the information values from the sequence of symbols by desymbolization.
20. The arithmetic decoder in accordance with claim 19, wherein the coding state of the arithmetic decoder is defined by the decoder-internal parameters which comprise an interval width parameter indicating a width of the current interval and a pointer pointing into the current interval and the arithmetic decoder is configured to derive from the entry point information a starting value of the pointer using which the arithmetically decoding the bitstream from the predetermined entry point onwards is commenced.
21. The arithmetic decoder in accordance with claim 19, wherein the coding state of the arithmetic decoder is defined by the decoder-internal parameters which comprise an interval width parameter indicating a width of the current interval and a pointer pointing into the current interval and the arithmetic decoder is configured to derive from the entry point information a starting value of the interval width parameter using which the arithmetically decoding the bitstream from the predetermined entry point onwards is commenced.
22. The arithmetic decoder in accordance with claim 19, wherein the coding state of the arithmetic decoder is defined by the decoder-internal parameters which comprise an interval width parameter

indicating a width of the current interval and a pointer pointing into the current interval and the arithmetic decoder is configured to derive from the entry point information a starting value of the pointer using which the arithmetically decoding the bitstream from the predetermined entry point onwards is commenced and to set the value of the interval width parameter comprised by the information on the coding state of the arithmetic decoder to be equal to a predetermined value and use the predetermined value for the interval width parameter for resuming the arithmetic decoding the sequence of symbols from the predetermined entry point onwards.

23. The arithmetic decoder in accordance with claim 22, wherein the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward and the arithmetic decoder is configured to preliminarily interrupting the arithmetic decoding the sequence of symbols at a subsequent predetermined entry point by arithmetically decoding a symbol of a predetermined symbol value immediately ahead the subsequent predetermined entry point before resuming the arithmetic decoding the sequence of symbols from the subsequent predetermined entry point onwards.

24. The arithmetic decoder in accordance with claim 19, configured to derive from the entry point information a bitstream pointer to a predetermined bit in the bitstream and use the predetermined bit as a bit to be read next after resuming the arithmetic decoding the bitstream from the predetermined entry point onwards.

25. The arithmetic decoder in accordance with claim 19, wherein the arithmetic decoder is configured to use context-adaptive arithmetic decoding for arithmetically decoding the sequence of symbols, comprising selecting, for a context-adaptively decoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols, wherein the entry point information is indicative of, for each of a set of one or more predetermined context models, a predetermined probability estimate for the respective predetermined context model, and the arithmetic decoder is configured to use the predetermined probability estimate in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model.

26. The arithmetic decoder in accordance with claim 19, wherein the arithmetic decoder is configured to use context-adaptive arithmetic decoding for arithmetically decoding the sequence of symbols, comprising selecting, for a context-adaptively decoded symbol of the sequence of symbols, a context model out of a plurality of context models, each of which has a probability estimate associated therewith, and adapting the probability estimates of the plurality of context models to actual symbol statistics using previously decoded symbols of the sequence of symbols, wherein the arithmetic decoder is configured to, for each of a set of one or more predetermined context models, set the probability estimate for the respective predetermined context model at the predetermined entry point to a default state and the arithmetic decoder is configured to use the default state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model.

27. The arithmetic decoder in accordance with claim 26, wherein the entry point information allows for resuming arithmetic decoding the bitstream from more than one entry point onward and the arithmetic decoder is configured to for each of the set of one or more predetermined context models, set the probability estimate for the respective predetermined context model at a subsequent predetermined entry point to a saved state manifesting itself at a predetermined condition during arithmetically decoding the sequence of symbols ahead the subsequent predetermined entry point and the arithmetic decoder is configured to use the saved state in resuming the adaptation of the probability estimates of the plurality of context models with respect to the respective predetermined context model from the subsequent predetermined entry point onwards.

28. The arithmetic decoder in accordance with claim 19, wherein the arithmetic decoder is

configured to use dependent dequantization for deriving from the sequence of information values a sequence of quantized values using a state machine, derive from the entry point information a quantization state starting with which the dependent quantization is resumed from the predetermined entry point onwards.

29. The arithmetic decoder in accordance with claim 19, wherein the symbols are bins and the desymbolization is a debinarization.

30. The arithmetic decoder in accordance with claim 19, wherein the information values is a sequence of syntax elements which represent a video.

31. The arithmetic decoder in accordance with claim 19, wherein the information values are neural network parameters.

32. The arithmetic decoder in accordance with claim 19, wherein the entry point information is comprised by a header preceding the bitstream, following the bitstream or inserted in between the bitstream.

33. The arithmetic decoder in accordance with claim 19, configured to use the entry point information so as to determine a starting version of the coding state of the arithmetic decoder and use the starting state for commencing arithmetically decoding the bitstream from the predetermined entry point onwards.

34. A method of arithmetically decoding a sequence of information values from a bitstream, comprising: deriving from the bitstream entry point information; using the entry point information so as to resume arithmetically decoding the bitstream from a predetermined entry point onward by arithmetically decoding a sequence of symbols from the bitstream by, for each symbol of the bitstream: determining, a current version of a coding state of an arithmetic decoder, a subinterval out of a plurality of subintervals into which a current interval is subdivided according to a probability estimate for the respective symbol, and deducing, based on the selected subinterval, a symbol value of the respective symbol, and renormalizing and updating decoder-internal parameters which define the coding state by use of the bitstream and the selected subinterval so as to acquire an updated version of the coding state of the arithmetic decoder for decoding a next symbol of the sequence of symbols, and deriving the information values from the sequence of symbols by desymbolization.

35. A non-transitory digital storage medium having a computer program stored thereon to perform the method of arithmetically encoding a sequence of information values into an arithmetic coded bitstream, comprising: symbolizing the information values into symbol strings so as to acquire a sequence of symbols; arithmetically encoding the sequence of symbols by subdividing a current interval, which defines a current version of a coding state of an arithmetic encoder, for each symbol, according to a probability estimate for the respective symbol, selecting a subinterval out of a plurality of the subintervals according to a symbol value of the respective symbol so as to acquire an updated version of the coding state of the arithmetic encoder, defined by the selected subinterval, for encoding a next symbol of the sequence of symbols, and renormalizing encoder-internal parameters which define the coding state under continuing the bitstream, providing the bitstream with entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward, when said computer program is run by a computer.

36. A non-transitory digital storage medium having a computer program stored thereon to perform the method of arithmetically decoding a sequence of information values from a bitstream, comprising: deriving from the bitstream entry point information; using the entry point information so as to resume arithmetically decoding the bitstream from a predetermined entry point onward by arithmetically decoding a sequence of symbols from the bitstream by, for each symbol of the bitstream: determining, based on a current version of a coding state of an arithmetic decoder, a selected subinterval out of a plurality of subintervals into which a current interval is subdivided according to a probability estimate for the respective symbol, and deducing, based on the selected subinterval, a symbol value of the respective symbol, and renormalizing and updating decoder-

internal parameters which define the coding state by use of the bitstream and the selected subinterval so as to acquire an updated version of the coding state of the arithmetic decoder for decoding a next symbol of the sequence of symbols, and deriving the information values from the sequence of symbols by desymbolization, when said computer program is run by a computer.

37. A method of arithmetically encoding a sequence of information values into an arithmetic coded bitstream, comprising: symbolizing the information values into symbol strings so as to acquire a sequence of symbols; arithmetically encoding the sequence of symbols by subdividing a current interval, which defines a current version of a coding state of an arithmetic encoder, for each symbol, according to a probability estimate for the respective symbol, into a plurality of the subintervals, selecting a subinterval out of the plurality of the subintervals according to a symbol value of the respective symbol so as to acquire an updated version of the coding state of the arithmetic encoder, defined by the selected subinterval, for encoding a next symbol of the sequence of symbols, and renormalizing encoder-internal parameters which define the coding state under continuing the bitstream, providing the bitstream with entry point information allowing for resuming arithmetic decoding the bitstream from a predetermined entry point onward.
